ZÜRCHER HOCHSCHULE FÜR ANGEWANDTE WISSENSCHAFTEN
DEPARTMENT LIFE SCIENCES UND FACILITY MANAGEMENT
INSTITUT FÜR COMPUTATIONAL LIFE SCIENCES (ICLS)

Bayesian Optimization and Reinforcement Learning for Capacity Optimization and Dispatching in a Discrete-Event Simulation of a Testing Laboratory

Bachelor Thesis

by

Fabian Frey

Bachelor's Degree Program Applied Digital Life Sciences 2023

Field of Study: Digital Labs and Production

Submission Date: 02.07.2026

Supervisor 1:
Marco Brunschwiler
ZHAW Life Sciences und Facility Management
Institute of Computational Life Sciences (ICLS)
Schloss 1
8820 Wädenswil
marco.brunschwiler@zhaw.ch

Supervisor 2:
Prof. Dr. Lukas Hollenstein
ZHAW Life Sciences und Facility Management
Institute of Computational Life Sciences (ICLS)
Schloss 1
8820 Wädenswil
lukas.hollenstein@zhaw.ch

Abstract

Dynamic laboratory systems can be characterized by stochastic arrivals, limited resources, queues, and repeated operational decisions. This thesis investigates how Bayesian Optimization and Q-learning-based Reinforcement Learning can be applied and evaluated for different optimization tasks within a discrete-event simulation of a testing laboratory. Bayesian Optimization was used to optimize static station and worker capacity configurations based on a weighted cost function, and its performance was compared with Random Search under default and changed objective weights. In a separate Reinforcement Learning simulation variant, a tabular Q-learning agent selected among four predefined dispatching-rule actions at the Sorting station during runtime: First In First Out, Earliest Due Date, Longest Waiting Time, and Highest Lateness Risk. Bayesian Optimization was additionally applied to tune selected Q-learning hyperparameters.

The Bayesian Optimization experiments showed that the method could identify high-performing capacity configurations within the evaluated search budget. Compared with Random Search, Bayesian Optimization achieved lower final best objective values descriptively under the default objective weights, although the difference was not statistically significant at the 5% level. Under the changed objective weights, Bayesian Optimization significantly outperformed Random Search. The change in objective weights also led to substantially different selected capacity configurations, indicating that the optimization outcome strongly depends on the formulation of the objective function. In the Reinforcement Learning experiments, Q-learning outperformed the random agent and several fixed dispatching-rule baselines, but did not clearly surpass the strongest heuristic baseline, Highest Lateness Risk. Bayesian Optimization-based hyperparameter tuning significantly improved Q-learning compared with the manually configured agent, although the practical effect on the independent holdout evaluation was small.

Overall, the results indicate that Bayesian Optimization and Q-learning can support complementary roles in simulation-based optimization. Bayesian Optimization is suitable for expensive black-box parameter search, such as capacity optimization and Reinforcement Learning hyperparameter tuning. Q-learning can support adaptive dispatching decisions, but its benefit depends strongly on the state representation, action space, reward design, and benchmark heuristics. The findings are limited to the implemented simulation models and should therefore be interpreted as simulation-based evidence rather than directly transferable recommendations for a real testing laboratory.

Abstract German

Dynamische Laborsysteme können durch stochastische Auftragseingänge, begrenzte Ressourcen, Warteschlangen und wiederkehrende operative Entscheidungen charakterisiert werden. Diese Arbeit untersucht, wie Bayesian Optimization und Q-learning-basiertes Reinforcement Learning für unterschiedliche Optimierungsaufgaben innerhalb einer diskreten Ereignissimulation eines Testlabors angewendet und evaluiert werden können. Bayesian Optimization wurde verwendet, um statische Kapazitätskonfigurationen von Stationen und Mitarbeitenden auf Basis einer gewichteten Kostenfunktion zu optimieren. Die Leistung wurde mit Random Search unter standardmässigen und veränderten Zielgewichtungen verglichen. In einer separaten Reinforcement-Learning-Simulationsvariante wählte ein tabellarischer Q-learning-Agent während der Simulationslaufzeit an der Sorting-Station zwischen vier vordefinierten Dispatching-Regel-Aktionen: First In First Out, Earliest Due Date, Longest Waiting Time und Highest Lateness Risk. Zusätzlich wurde Bayesian Optimization eingesetzt, um ausgewählte Q-learning-Hyperparameter zu optimieren.

Die Bayesian-Optimization-Experimente zeigten, dass die Methode innerhalb des untersuchten Suchbudgets leistungsfähige Kapazitätskonfigurationen identifizieren konnte. Im Vergleich zu Random Search erzielte Bayesian Optimization unter den standardmässigen Zielgewichtungen deskriptiv niedrigere finale Bestwerte der Zielfunktion, wobei der Unterschied auf dem 5%-Niveau nicht statistisch signifikant war. Unter den veränderten Zielgewichtungen schnitt Bayesian Optimization signifikant besser ab als Random Search. Die Veränderung der Zielgewichtungen führte zudem zu deutlich unterschiedlichen ausgewählten Kapazitätskonfigurationen, was darauf hindeutet, dass das Optimierungsergebnis stark von der Formulierung der Zielfunktion abhängt. In den Reinforcement-Learning-Experimenten übertraf Q-learning den Random Agent sowie mehrere feste Dispatching-Regel-Baselines, konnte jedoch die stärkste heuristische Baseline, Highest Lateness Risk, nicht klar übertreffen. Das auf Bayesian Optimization basierende Hyperparameter-Tuning verbesserte Q-learning signifikant gegenüber dem manuell konfigurierten Agenten, obwohl der praktische Effekt in der unabhängigen Holdout-Evaluation klein war.

Insgesamt deuten die Ergebnisse darauf hin, dass Bayesian Optimization und Q-learning komplementäre Rollen in der simulationsbasierten Optimierung unterstützen können. Bayesian Optimization eignet sich für aufwendige Black-Box-Parametersuchen, beispielsweise für die Kapazitätsoptimierung und das Hyperparameter-Tuning von Reinforcement Learning. Q-learning kann adaptive Dispatching-Entscheidungen unterstützen, sein Nutzen hängt jedoch stark von der Zustandsrepräsentation, dem Aktionsraum, dem Reward Design und den verwendeten heuristischen Vergleichsverfahren ab. Die Ergebnisse sind auf die implementierten Simulationsmodelle beschränkt und sollten daher als simulationsbasierte Evidenz und nicht als direkt übertragbare Empfehlungen für ein reales Testlabor interpretiert werden.

Contents

List of Figures

List of Tables

List of Abbreviations

ADLS	**A**pplied **D**igital **L**ife **S**ciences
BO	**B**ayesian **O**ptimization
EDD	**E**arliest **D**ue **D**ate
FIFO	**F**irst **I**n **F**irst **O**ut
HPC	**H**igh-**P**erformance **C**omputing
KPI	**K**ey **P**erformance **I**ndicator
RL	**R**einforcement **L**earning
SLURM	**S**imple **L**inux **U**tility for **R**esource **M**anagement

1 Introduction

Modern laboratory, production, and service systems are often characterized by stochastic arrivals, limited resources, queues, and dynamic process interactions. In such systems, operational decisions influence performance indicators such as throughput, waiting times, due-date reliability, and resource utilization. Analytical optimization of these decisions is often difficult because the underlying system behaviour may be nonlinear, stochastic, and only partially observable through simulation output. Discrete-event simulation therefore provides a useful experimental environment for evaluating alternative configurations and decision policies under controlled conditions. However, simulation alone does not determine which configuration or policy should be selected. This motivates the use of simulation-based optimization methods, where optimization algorithms use simulation output to search for improved operational decisions (Amaran et al., 2016; Gosavi, 2015).

1.1 Background and Motivation

Testing laboratories can be understood as operational process systems in which incoming orders or samples pass through several processing stages before they are completed and dispatched. Their performance is influenced by the interaction of arrival patterns, station capacities, worker availability, queues, routing decisions, and possible rework. As a result, improving such a system is not only a question of increasing individual capacities, but also of understanding how local decisions affect the overall flow of orders. This creates a trade-off between service-oriented objectives, such as high completion rates and low lateness, and resource-oriented objectives, such as avoiding unnecessarily high station or worker capacities.

Simulation provides a suitable environment for analysing these interactions because alternative configurations and decision policies can be evaluated without changing a real laboratory process. Greasley and Edwards (2021) describe discrete-event simulation as a mature operations research technique for process modelling and emphasize its role in evaluating possible process changes. However, a simulation model does not automatically determine which capacity setting or decision policy should be preferred. Once several decision variables, stochastic outputs, and conflicting objectives are involved, the simulation model becomes the basis for an optimization problem rather than a solution by itself.

This thesis therefore investigates two data-driven optimization approaches within a discrete-event simulation of a testing laboratory. Bayesian Optimization is motivated by the static configuration problem, where station and worker capacities can be evaluated through complete simulation runs and treated as an expensive black-box objective. This is consistent with the use of Bayesian Optimization for simulation-based optimization of discrete-event simulations described by Zmijewski and Meseth (2023). Q-learning is motivated by a different type of decision problem, because dispatching at the Sorting station requires repeated decisions under changing system states. In addition, Bayesian Optimization is used to tune selected Q-learning hyperparameters, which follows the general motivation of Barsce et al. (2018) that RL performance can depend strongly on hyperparameter settings. The two methods are therefore not treated as interchangeable algorithms, but as complementary approaches for different optimization roles within the same simulation-based case study.

1.2 Research Objective and Scope

The objective of this thesis is to apply and evaluate Bayesian Optimization and Q-learning-based Reinforcement Learning for different optimization tasks within a discrete-event simulation of a testing laboratory. The study focuses on how both approaches can support simulation-based operational decision-making under stochastic system behaviour. Bayesian Optimization and Q-learning are not evaluated as interchangeable algorithms for one identical optimization task. Instead, they are examined as complementary methods that operate at different decision levels of the same case study.

In the static optimization part, Bayesian Optimization is used to search for station and worker capacity settings. The resulting configurations are evaluated with respect to a weighted cost function and compared with Random Search under the same evaluation budget. The analysis also considers how changing the relative weighting of service-related and capacity-related objective components affects the selected configurations. In the dynamic optimization part, a Q-learning agent is used

to select dispatching rules at the Sorting station during the simulation run. The selected rule then determines which waiting order is processed next. Bayesian Optimization is additionally used as a meta-optimization method for tuning selected Q-learning hyperparameters.

The scope of this thesis is limited to a simulation-based case study and to the implemented model assumptions, objective definitions, reward formulation, and evaluation budgets. The work does not aim to validate an operational deployment in a real testing laboratory. It also does not aim to develop a general Automated Reinforcement Learning framework or to determine whether Bayesian Optimization or Reinforcement Learning is generally superior. Rather, the thesis investigates how both methods can be used in distinct but connected roles within a controlled discrete-event simulation environment.

1.3 Research Questions and Hypotheses

Based on the research objective and scope defined above, this thesis is guided by one main research question. The main research question is formulated as follows:

> **RQ:** How can Bayesian Optimization and Q-learning-based Reinforcement Learning be applied and evaluated for different optimization tasks within a discrete-event simulation of a testing laboratory?

To answer this main research question, four sub-questions are defined:

- **RQ1:** How effectively can Bayesian Optimization improve static simulation configurations, such as station capacities and worker capacity, with respect to a weighted cost function?
- **RQ2:** How does Bayesian Optimization compare to Random Search when optimizing the same simulation configuration problem under different objective weightings?
- **RQ3:** Can a Q-learning agent improve system performance through dynamic dispatching decisions at the Sorting station compared with fixed dispatching rules and a random dispatching agent?
- **RQ4:** To what extent can Bayesian Optimization improve the performance of the Q-learning agent through hyperparameter tuning?

Based on these research questions, the following hypotheses are evaluated:

- **H1:** Bayesian Optimization achieves lower final best objective values than Random Search under the same evaluation budget.
- **H2:** Changing the relative weights of order-related and capacity-related cost components leads Bayesian Optimization to identify different optimal simulation configurations.
- **H3:** A Q-learning agent that dynamically selects dispatching rules at the Sorting station achieves better simulation performance than fixed dispatching-rule baselines and a random dispatching agent.
- **H4:** Bayesian Optimization-based hyperparameter tuning improves the performance of the Q-learning agent compared with manually selected baseline hyperparameters.

RQ1 is addressed descriptively by analysing the configurations, objective values, and operational key performance indicators obtained through Bayesian Optimization. RQ2 is addressed through the comparison between Bayesian Optimization and Random Search and through the analysis of changed objective weightings. H1 and H2 operationalize these comparative and weighting-related aspects of RQ2. H3 addresses the comparison between Q-learning and the dispatching-rule baselines. H4 addresses the effect of Bayesian Optimization-based hyperparameter tuning on Q-learning performance.

The following chapter introduces the theoretical foundations of discrete-event simulation, simulation-based optimization, Bayesian Optimization, Reinforcement Learning, and their complementary use in this thesis.

2 Theoretical Background

This chapter introduces the theoretical concepts required to understand the simulation-based optimization approaches applied in this thesis. The work is based on a discrete-event simulation model that represents the operational behaviour of a testing laboratory and serves as an experimental environment for evaluating different optimization methods. The following sections therefore first introduce discrete-event simulation as the modelling foundation. Subsequently, scheduling and dispatching in dynamic systems, simulation-based optimization, Bayesian Optimization, Reinforcement Learning, and the interaction between Bayesian Optimization and Reinforcement Learning are discussed. The focus is on the methodological principles that are relevant for the later implementation and evaluation, rather than on providing a general state-of-research overview.

2.1 Discrete-Event Simulation

Discrete-event simulation (DES) is a modelling approach used to represent systems whose state changes at discrete points in simulated time. In contrast to continuous simulation, where the system state evolves continuously, DES advances from one event to the next. Such events may include the arrival of an order, the start or completion of a processing step, the allocation or release of a resource, or the movement of an entity from one queue to another. This makes DES particularly suitable for systems in which process flows, waiting times, limited capacities, and stochastic behaviour play an important role. Greasley and Edwards (2021) describe DES as a mature operations research technique for process modelling and emphasize its ability to support detailed and flexible evaluations of process changes.

A DES model is typically built from several basic elements. Entities represent the objects that move through the system, such as customers, jobs, patients, or orders. Resources represent limited capacities that are required to perform an activity, for example machines, workstations, operators, or laboratory stations. Queues occur when entities wait for a required resource or for the next process step to become available. Events define the points in time at which the state of the system changes. In addition, processing times, inter-arrival times, routing decisions, and outcome probabilities are often modelled using probability distributions. This allows the simulation to represent operational variability that would be difficult to capture analytically.

The event-based structure of DES is well suited for modelling production, logistics, laboratory, and service systems because such systems often consist of sequences of activities with constrained resources. In these settings, performance is not only determined by the average processing time of individual activities, but also by interactions between stations, queues, bottlenecks, and stochastic arrivals. For example, increasing the capacity of one station may not improve overall performance if another station remains the dominant bottleneck. DES enables such interactions to be analysed by executing the model under different configurations and observing the resulting performance indicators.

Since DES models commonly include stochastic inputs, their outputs are also stochastic. A single simulation run therefore represents only one possible realization of the system behaviour under a given configuration. Reliable performance estimation usually requires multiple replications with different random seeds, especially when comparing alternative configurations or policies. The resulting output data can then be analysed statistically, for example by comparing mean values, variability, or confidence intervals of relevant key performance indicators. This is important when simulation results are used as the basis for optimization, because the optimizer does not observe the true expected performance directly, but only estimates derived from finite simulation replications.

In the context of this thesis, DES provides the experimental environment in which order flows, station capacities, queues, processing times, and decision rules can be represented explicitly. The simulation model is implemented in Python using the salabim package. According to van der Ham (2023), salabim supports process-oriented discrete-event simulation through concepts such as components, resources, queues, monitors, and process interaction methods. These concepts are consistent with the modelling requirements of the laboratory case study, where orders move through several processing stages and compete for limited station capacities.

DES becomes particularly relevant when combined with optimization methods. Gosavi (2015) describes simulation-based optimization as the use of simulation models to solve stochastic opti-

mization problems related to discrete-event systems. In such settings, the simulation acts as an evaluation mechanism: different parameter settings or decision policies are applied to the model, and their performance is assessed through simulation output. This thesis follows this principle by using the DES model both as a black-box evaluation function for static simulation configurations and as an interactive environment for dynamic dispatching decisions.

2.2 Scheduling and Dispatching in Dynamic Systems

2.2.1 Scheduling as a Dynamic Decision Problem

Scheduling describes the allocation and sequencing of jobs, orders, or tasks on limited resources over time. In production, logistics, laboratory, and service systems, scheduling decisions influence operational performance indicators such as flow time, waiting time, resource utilization, throughput, and due-date reliability. A scheduling problem therefore does not only concern whether sufficient capacity exists, but also which waiting order should be processed next when a resource becomes available. In static scheduling problems, all relevant jobs and system information are assumed to be known before the schedule is created. In dynamic scheduling problems, this assumption is relaxed because new jobs may arrive during operation, processing times may vary, resources may become temporarily unavailable, and priorities may change over time. This makes dynamic scheduling more difficult than static planning because decisions must be made repeatedly under changing system conditions. Wang and Liao (2024) describe dynamic job shop scheduling as a setting in which manufacturing systems must react to uncertainty and random disturbances such as job arrivals, machine breakdowns, and due-date changes. Such characteristics are also relevant for simulation-based laboratory systems, where orders enter the system over time, move through multiple process stages, and compete for limited station capacities.

2.2.2 Dispatching Rules

Dispatching rules are commonly used as simple and fast decision mechanisms for online scheduling. A dispatching rule is applied when a resource becomes available and multiple waiting jobs are eligible for processing. The rule assigns a priority value to each waiting job, and the job with the highest priority is selected for processing. Freitag and Hildebrandt (2016) describe dispatching rules as part of the operational control of manufacturing systems, where waiting jobs are evaluated whenever a machine becomes idle. The advantage of dispatching rules is that they are computationally inexpensive, interpretable, and directly applicable in real-time decision situations. This makes them useful in complex and stochastic systems where repeatedly computing a complete optimal schedule is not feasible. Zhang et al. (2009) argue that dispatching rules are widely used in complex manufacturing systems because analytical optimization can be too difficult or too slow for real-time operation. However, the same authors also emphasize that selecting suitable dispatching rules is itself a challenging problem. Several basic dispatching principles are relevant in this thesis.

- First-In-First-Out (FIFO) prioritizes the order that entered the queue first.
- Earliest Due Date (EDD) prioritizes the order with the earliest absolute due date.
- Longest Waiting Time (LWT) prioritizes the order with the longest current waiting time in the queue.
- Lateness Risk prioritizes orders according to their risk of becoming late.

FIFO represents a simple queue-order-based rule and does not explicitly consider due dates. EDD is a due-date-oriented rule and therefore gives priority to orders with stricter deadlines. LWT focuses on the time an order has already spent waiting and can help avoid excessive waiting for individual orders. Lateness Risk extends due-date-oriented prioritization by considering how close an order is to becoming late, rather than only comparing due dates directly. These rules represent different operational priorities and can therefore lead to different system behaviour under the same simulation conditions.

2.2.3 Limits of Fixed Dispatching Rules

Although dispatching rules are practical, their performance often depends on the current state of the system. A rule that performs well under low congestion may not perform well under high congestion. Similarly, a rule that reduces average waiting time may not necessarily minimize lateness or improve due-date reliability. This creates a trade-off between simplicity and adaptability.

Rule-based control is attractive because it is transparent and easy to implement, but a single fixed rule may be too rigid for dynamic systems with changing workloads and bottlenecks. Haeussler and Netzer (2020) distinguish between rule-based and optimization-based workload control concepts and show that the relative performance of such approaches can depend on the demand and utilization scenario. This supports the general argument that operational control strategies should be evaluated in the context of the specific system and objective function. In simulation studies, dispatching rules can be tested under controlled stochastic conditions without interfering with a real operational system. This makes discrete-event simulation a suitable environment for evaluating and comparing rule-based scheduling decisions. It also creates the basis for adaptive approaches in which the decision mechanism changes depending on the observed system state.

2.2.4 Adaptive Rule Selection

Instead of applying one fixed dispatching rule throughout the complete simulation run, an adaptive decision mechanism can select different rules in different situations. This idea preserves the interpretability of dispatching rules while allowing the decision policy to react to the current state of the system. For example, a policy may prefer queue-order-based prioritization under normal workload conditions but switch to due-date-oriented prioritization when many orders are close to their deadlines. Recent scheduling studies increasingly formulate such adaptive decision problems as reinforcement learning problems. In this formulation, the scheduling system is represented as an environment, and the learning agent selects actions based on the observed system state. Wang and Liao (2024) use a discrete-event simulation environment to train a reinforcement learning agent for dynamic job shop scheduling and design actions and rewards around scheduling decisions at decision points. In the context of this thesis, the agent does not select an individual order directly. Instead, it selects one of several predefined and interpretable dispatching rules whenever a decision point occurs at the controlled station. This keeps the action space small, supports comparison with fixed dispatching-rule baselines, and makes the learned policy easier to interpret.

2.3 Simulation-Based Optimization

2.3.1 Simulation as an Optimization Environment

Discrete-event simulation can be used not only to analyse a system, but also to evaluate alternative system configurations and decision strategies. In this case, the simulation model becomes an experimental environment in which different inputs can be tested without changing the real system. The input to the simulation may include fixed configuration parameters, such as resource capacities or buffer sizes, or decision rules that influence the behaviour of the system during runtime. The output of the simulation consists of performance indicators such as throughput, waiting time, flow time, tardiness, resource utilization, or cost.

Simulation-based optimization uses this input-output relationship to search for improved system configurations or policies. Instead of deriving an analytical objective function, the optimizer evaluates candidate solutions by executing the simulation model. This is useful when the system is too complex, stochastic, or nonlinear to be described by a closed-form mathematical model. Amaran et al. (2016) define simulation optimization as the optimization of an objective function subject to possible constraints, where both objective and constraints can be evaluated through stochastic simulation.

2.3.2 Black-Box Objective Functions and Stochastic Output

In many simulation-based optimization problems, the simulation model is treated as a black-box function. This means that the optimizer can select an input configuration and observe the resulting output, but it does not require direct access to an analytical expression of the objective function. Amaran et al. (2016) emphasize that simulation optimization differs from algebraic model-based mathematical programming because the simulation may only provide input-output observations. This also implies that derivative information is often unavailable or unreliable, especially when the output is noisy.
A general simulation-based optimization problem can be written as:

$$x^* = \arg\min_{x \in \mathcal{X}} \mathrm{E}_\omega[f(x, \omega)] \tag{1}$$

Here, x denotes a candidate configuration or decision vector, and $\mathcal{X}$ denotes the feasible search space. The term ω represents the stochastic influences within a simulation run, such as random arrivals, processing times, routing outcomes, or other uncertain events. The function $f(x, \omega)$ represents the observed performance or cost of configuration x under one realization of the stochastic simulation. Since the true expected value $\mathrm{E}_\omega[f(x, \omega)]$ is usually unknown, it must be estimated from a finite number of simulation replications.

The stochastic nature of simulation output creates an additional challenge for optimization. Two simulation runs with the same input configuration may lead to different objective values because the random events in the simulation differ. As a result, optimization algorithms must search under uncertainty and may need to balance solution quality with the number of replications used for evaluation. This is especially relevant when each simulation run is computationally expensive, because increasing the number of replications improves the reliability of the estimate but also increases the total optimization cost.

2.3.3 Parametric and Control Optimization

Gosavi (2015) distinguishes between parametric optimization and control optimization in the context of simulation-based optimization. Parametric optimization refers to static optimization problems in which the goal is to find suitable values for a set of parameters. These parameters are typically chosen before the simulation run starts and remain fixed during the simulation. Examples include resource capacities, processing settings, reorder levels, buffer sizes, or other structural configuration variables.

Control optimization refers to dynamic optimization problems in which the goal is to determine suitable actions while the system evolves over time. In this case, the optimizer does not only select one fixed parameter vector, but learns or defines a policy that maps observed system states to actions. This type of optimization is relevant for routing, dispatching, sequencing, and other operational decisions that must be made repeatedly during system operation. Control optimization is therefore closely related to sequential decision-making and provides the conceptual link between simulation-based optimization and reinforcement learning.

The distinction between parametric and control optimization is important because both problem types require different algorithmic approaches. Parametric optimization can often be treated as a black-box search problem, where complete simulation runs are evaluated under different parameter settings. Control optimization requires interaction with the system during the simulation, because the quality of a decision depends on the current state and on its long-term consequences. This difference explains why methods such as Bayesian Optimization and Reinforcement Learning can both be used in simulation-based optimization, but for different optimization roles.

2.3.4 Relevance for This Thesis

This thesis uses the discrete-event simulation model as the common evaluation environment for several optimization tasks. Bayesian Optimization is used for parametric optimization by searching for static simulation configurations, such as station and worker capacities. The performance of each configuration is evaluated by running the simulation and calculating a weighted cost function from the resulting performance indicators. This corresponds to a black-box simulation optimization problem, because the optimizer observes objective values generated by the simulation but does not use an analytical model of the underlying system.

Reinforcement Learning is used for control optimization by learning dynamic dispatching decisions during the simulation run. In this case, the agent observes the current system state at decision points and selects one of several dispatching rules as an action. The quality of the policy is then evaluated through the rewards and performance indicators generated by the simulation. This differs from the static Bayesian Optimization setting because decisions are made repeatedly within the simulation rather than only before the simulation starts.

Bayesian Optimization is also used as a meta-optimization method for tuning the hyperparameters of the Reinforcement Learning agent. This creates a second parametric optimization problem, where the parameters being optimized are not simulation capacities but learning-related settings of the agent. Zmijewski and Meseth (2023) describe Bayesian Optimization as a promising approach

for simulation-based optimization because it is designed for expensive and noisy black-box functions. In this thesis, this principle is applied both to the optimization of simulation configurations and to the tuning of the learning process.

2.4 Bayesian Optimization

2.4.1 Black-Box Optimization

Bayesian Optimization is a sequential optimization method for finding good solutions of expensive black-box objective functions. A function is treated as a black box when its analytical form is unknown or unavailable, but input configurations can be evaluated and the resulting objective values can be observed. This situation is common in simulation-based optimization, where a candidate configuration must be passed to a simulation model before its performance can be measured. It is also common in hyperparameter optimization, where a candidate parameter setting must be evaluated by training or running a model before its performance is known.
The general optimization problem can be written as:

$$x^* = \arg\min_{x \in \mathcal{X}} f(x) \tag{2}$$

Here, x denotes a candidate configuration, $\mathcal{X}$ denotes the search space, and $f(x)$ denotes the unknown objective function. In practice, the true value of $f(x)$ may not be observed directly because evaluations can be affected by stochastic noise. A noisy observation can be represented as:

$$y_i = f(x_i) + \epsilon_i \tag{3}$$

Here, y_i is the observed objective value at configuration x_i, and ϵ_i represents observation noise. In the context of stochastic simulation, this noise can result from random arrivals, random processing times, routing probabilities, or other stochastic mechanisms in the model. Liu (2023) describes Bayesian Optimization as a framework for global optimization in unknown and potentially noise-corrupted environments, where the optimizer must use a limited evaluation budget efficiently. This makes Bayesian Optimization relevant when exhaustive search, grid search, or repeated manual tuning would require too many evaluations.

2.4.2 Surrogate Modelling with Gaussian Processes

The central idea of Bayesian Optimization is to replace the expensive objective function with a cheaper probabilistic surrogate model. The surrogate model is fitted to the configurations that have already been evaluated and is then used to reason about promising regions of the search space. Unlike a deterministic regression model, the surrogate model in Bayesian Optimization should not only predict expected objective values, but also quantify uncertainty in unexplored regions.

Gaussian Processes are commonly used as surrogate models in Bayesian Optimization. A Gaussian Process defines a probability distribution over functions and can be written as:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')) \tag{4}$$

In this expression, $m(x)$ is the mean function and $k(x, x')$ is the kernel function. The mean function represents the expected function value before or after observing data. The kernel function describes the similarity between two input points and determines how information from evaluated configurations generalizes to unevaluated configurations. After each new observation, the Gaussian Process posterior is updated and provides a predictive distribution for each candidate configuration.

This predictive distribution usually consists of a predicted mean and a predictive uncertainty. The predicted mean indicates how good a candidate is expected to be according to the current model. The uncertainty indicates how little is known about that candidate based on the available observations. This uncertainty estimate is important because it allows Bayesian Optimization to search not only where good objective values are expected, but also where additional information may be valuable. Liu (2023) identifies the Gaussian Process and the acquisition function as two central components of the Bayesian Optimization workflow.

2.4.3 Acquisition Functions

The acquisition function determines which candidate configuration should be evaluated next. It uses the surrogate model to assign a utility value to candidate points in the search space. The next evaluation point is then selected by optimizing the acquisition function:

$$x_{t+1} = \arg\max_{x \in \mathcal{X}} \alpha(x) \tag{5}$$

Here, $\alpha(x)$ denotes the acquisition function, and x_{t+1} denotes the next configuration to be evaluated. The acquisition function is usually much cheaper to evaluate than the original objective function because it is computed from the surrogate model rather than from the expensive simulation or training process. This creates an inner optimization problem that guides the outer optimization loop.

A key role of the acquisition function is to balance exploitation and exploration. Exploitation means selecting configurations that are expected to perform well according to the current surrogate model. Exploration means selecting configurations in uncertain regions of the search space, where the model may gain useful information from an additional evaluation. If the optimizer focuses too strongly on exploitation, it may converge prematurely to a local optimum. If it focuses too strongly on exploration, it may spend too much of the evaluation budget on unpromising regions. Common acquisition functions include Expected Improvement, Probability of Improvement, and Upper Confidence Bound. These functions differ in how they combine predicted performance and uncertainty, but they all serve the same general purpose of guiding sequential sampling decisions under uncertainty.

The Bayesian Optimization loop therefore consists of repeated model updating and candidate selection. First, one or more initial configurations are evaluated. Second, a surrogate model is fitted to the observed configurations and objective values. Third, an acquisition function is optimized to propose the next candidate configuration. Finally, the proposed candidate is evaluated using the true expensive objective function, and the new observation is added to the dataset. This loop continues until the evaluation budget or another stopping criterion is reached.

2.4.4 Bayesian Optimization for Simulation-Based Optimization

Bayesian Optimization is well suited for simulation-based optimization because simulation models often satisfy the assumptions of expensive black-box functions. The objective value of a candidate configuration is usually only available after the simulation has been executed. Furthermore, stochastic simulation output may vary across replications, meaning that the optimizer must work with noisy estimates of performance. As discussed by Amaran et al. (2016), simulation optimization problems frequently involve expensive evaluations, stochastic outputs, and unavailable derivative information. Bayesian Optimization addresses these difficulties by using previous evaluations to build a probabilistic model of the objective function and by selecting new evaluations in a sample-efficient way.

Zmijewski and Meseth (2023) specifically discuss Bayesian Optimization in the context of discrete-event simulation and present SimBO as a framework for simulation-based optimization using sequential optimization algorithms. They argue that Bayesian Optimization can be useful for discrete-event simulation problems because many simulation evaluations may be computationally expensive and because fast optimization is important when simulation-based optimization is intended to support operational decision-making. This is consistent with the use of Bayesian Optimization in this thesis, where each evaluated configuration requires multiple simulation replications to estimate its objective value.

In this thesis, Bayesian Optimization is relevant as a sample-efficient method for expensive and stochastic simulation evaluations, both for static configuration search and for later RL hyperparameter tuning. The concrete positioning of BO together with RL is summarized in Section 2.6.4, while the detailed search spaces and objective definitions are described in the methodology chapter.

2.5 Reinforcement Learning

2.5.1 Sequential Decision-Making and Agent-Environment Interaction

Reinforcement Learning (RL) is a learning approach for sequential decision-making problems in which an agent learns by interacting with an environment. At each decision step, the agent observes the current state of the environment, selects an action, and receives a reward signal as feedback. Based on this feedback, the agent updates its behaviour with the objective of maximizing long-term cumulative reward rather than only optimizing the immediate reward of a single action. Afshar et al. (2026) describe RL as an approach in which an agent learns an optimal behaviour, or policy, through repeated interaction with its environment.

This interaction-based learning principle distinguishes RL from supervised learning. In supervised learning, the learning algorithm is usually trained on labelled examples that specify the desired output for a given input. In RL, such labels are not directly available because the quality of an action depends on its consequences over time. The agent therefore needs to learn from delayed and potentially noisy feedback. This makes RL suitable for operational decision problems in which actions influence not only the current system state, but also future system performance.

In this thesis, the environment is represented by the discrete-event simulation model. The agent interacts with this environment at predefined decision points and receives feedback based on the consequences of its dispatching decisions. This makes RL a suitable method for the control optimization part of the simulation-based optimization problem, because the decision policy is learned through repeated simulation runs rather than derived from an explicit analytical model.

2.5.2 Markov Decision Process

RL problems are commonly formulated as Markov Decision Processes (MDPs). An MDP provides a formal representation of a sequential decision-making problem and can be written as:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) \tag{6}$$

Here, $\mathcal{S}$ denotes the state space and contains all possible states that the agent may observe. The action space $\mathcal{A}$ contains all actions available to the agent. The transition function P describes the probability of moving from one state to another after selecting a given action. The reward function R defines the immediate feedback received after an action. The discount factor γ determines how strongly future rewards are considered relative to immediate rewards.

The Markov property assumes that the current state contains all relevant information required for predicting the future development of the system. In other words, the next state depends on the current state and action, but not on the full history of previous states and actions. In practical simulation applications, this assumption is an approximation because the state representation may not capture every detail of the simulated system. Nevertheless, the MDP framework provides a useful structure for defining the state variables, action set, reward signal, and learning objective of an RL problem.

Gosavi (2015) discusses MDPs and reinforcement learning in the context of simulation-based control optimization. In production scheduling, Yoo et al. (2025) describe the MDP formulation through state spaces, action spaces, transition probabilities, reward functions, and discount factors. They also emphasize that transition dynamics in scheduling environments do not always need to be explicitly defined, because the agent can learn effective strategies through interaction with the environment.

In the context of this thesis, the state represents selected information about the current condition of the simulated laboratory system at the decision point. The action represents the selection of a dispatching rule. The reward provides feedback on the quality of the decision. The transition is generated by the simulation after the selected dispatching rule has been applied. The policy defines how the agent selects dispatching rules under different observed system states.

2.5.3 Q-Learning

Q-learning is a model-free and value-based RL algorithm. It is called model-free because the agent does not need to know the transition probabilities of the environment. It is value-based because the agent learns an action-value function that estimates the expected long-term value of selecting a given action in a given state. This action-value function is denoted as $Q(s, a)$.

The value $Q(s, a)$ represents the expected return when the agent selects action a in state s and then continues according to its learned behaviour. The agent updates the Q-values from observed transitions during interaction with the environment. The standard Q-learning update rule can be written as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \tag{7}$$

Here, s_t and a_t denote the state and action at time step t. The term r_{t+1} denotes the reward received after taking action a_t. The parameter α is the learning rate and determines how strongly new information changes the existing Q-value. The discount factor γ determines the importance of future rewards. The term $\max_{a'} Q(s_{t+1}, a')$ represents the estimated value of the best action available in the next state. The expression inside the brackets is the temporal-difference error, which measures the difference between the current Q-value and the newly observed target value.

Gosavi (2015) presents Q-learning as a reinforcement learning method for control optimization in stochastic systems. For the problem considered in this thesis, Q-learning is appropriate because the state and action spaces are deliberately kept discrete and interpretable. The agent does not need to approximate a high-dimensional continuous policy. Instead, it learns a table of Q-values for combinations of discrete system states and predefined dispatching-rule actions.

2.5.4 Exploration and Exploitation

A central challenge in RL is the balance between exploration and exploitation. Exploitation means selecting the action that currently appears to be best according to the learned Q-values. Exploration means selecting alternative actions to gather additional information about their consequences. Both behaviours are necessary during learning. If the agent exploits too early, it may converge to a suboptimal policy. If it explores too much, it may fail to consistently use actions that have already been identified as beneficial.

A common strategy for balancing exploration and exploitation in Q-learning is the ϵ-greedy policy. Under this strategy, the agent selects a random action with probability ϵ and selects the action with the highest current Q-value with probability $1 - \epsilon$:

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \epsilon, \\ \arg\max_a Q(s_t, a), & \text{with probability } 1 - \epsilon. \end{cases} \tag{8}$$

The value of ϵ can be kept constant or reduced over time. A decreasing ϵ value supports stronger exploration in early learning phases and stronger exploitation after the agent has collected more experience. This is useful in simulation-based learning because the agent can initially test different dispatching rules under many system states before gradually relying more on the best observed actions.

2.5.5 Deep Reinforcement Learning as Context

Deep Reinforcement Learning (DRL) extends RL by using neural networks as function approximators. This is especially useful when the state space is high-dimensional, continuous, or too large to be represented in a table. Instead of storing one Q-value for every state-action pair, a neural network can approximate value functions or policies from input features. This allows DRL methods to handle more complex environments than tabular RL methods.

Several DRL approaches have been applied to scheduling problems. Value-based methods, such as Deep Q-Networks, extend Q-learning by approximating the Q-function with a neural network. Policy-based methods directly learn a policy that maps states to actions. Actor-critic methods

combine value estimation and policy learning. Yoo et al. (2025) describe value-based and policy-based DRL paradigms in production scheduling and relate them to different types of action spaces. Shi et al. (2020) apply deep reinforcement learning to scheduling in discrete automated production lines and emphasize that DRL can be useful when state representations are difficult to define manually.

Although DRL is relevant for complex scheduling environments, this thesis uses tabular Q-learning rather than a neural-network-based RL algorithm. This choice is consistent with the deliberately limited and discrete state-action representation used in the simulation model. The action space consists of a small number of predefined dispatching rules, and the state representation is discretized. This keeps the learning process transparent and makes the resulting policy easier to interpret. It also reduces implementation complexity compared with DRL methods, which require additional design decisions related to network architecture, training stability, and hyperparameter tuning.

2.5.6 Reinforcement Learning in DES-Based Scheduling

Discrete-event simulation provides a suitable environment for training and evaluating RL agents in scheduling problems. The agent can interact with a simulated system repeatedly without disturbing real operations. This is important because RL agents often require many interactions before a useful policy is learned. Simulation therefore provides a safe and controllable setting in which different decision policies can be tested under stochastic operating conditions.

Several studies combine DES and RL for dynamic scheduling. Rabe et al. (2017) combine a discrete-event simulation model of a logistics network with a deep reinforcement learning agent that learns actions affecting costs and service level. Shi et al. (2020) build a DES environment for deep reinforcement learning in discrete automated production line scheduling. Wang and Liao (2024) train a reinforcement learning scheduling agent in an integrated DES and DRL framework for dynamic job shop scheduling, where states, actions, and rewards are defined at scheduling decision points. These studies illustrate that DES can serve as the operational environment in which RL agents learn adaptive decision policies.

In this thesis, RL is used as an online control mechanism for adaptive dispatching decisions at the Sorting station. Its concrete role in relation to Bayesian Optimization is positioned in Section 2.6.4.

2.6 Interplay Between Bayesian Optimization and Reinforcement Learning

2.6.1 Complementary Optimization Roles

Bayesian Optimization and Reinforcement Learning address different types of optimization problems. Bayesian Optimization is mainly used for sample-efficient optimization of expensive black-box functions. It evaluates complete candidate configurations, updates a probabilistic surrogate model, and then proposes the next configuration to test. This makes BO suitable for static parameter optimization and for meta-optimization tasks where each evaluation is computationally expensive.

Reinforcement Learning, in contrast, is used for sequential decision-making problems. An RL agent interacts with an environment, observes states, selects actions, and learns a policy from reward feedback. The objective is not only to find one fixed parameter setting, but to learn a decision strategy that can react to changing system states. This makes RL suitable for dynamic control problems, such as adaptive dispatching or routing decisions in a simulation environment.

The two methods can therefore complement each other. BO can be used when the design or configuration of an RL system itself becomes an expensive black-box optimization problem. This includes the optimization of policy parameters, hyperparameters, reward-function weights, or state representations. RL can then use the optimized configuration to learn or improve a dynamic decision policy. In this sense, BO can operate at a higher optimization level, while RL operates at the level of sequential decision-making within the environment.

2.6.2 Bayesian Optimization for Policy Search

One direct way to combine BO and RL is to use BO for policy search. In this approach, a policy is described by a vector of parameters. BO proposes a candidate policy parameterization, the policy is executed in an environment, and the observed return is used as the objective value. The next policy parameterization is then selected based on the surrogate model and acquisition function.

Wilson et al. (2014) describe this setting by applying Bayesian Optimization to parametric policies in reinforcement learning. They argue that BO is attractive for policy search because it uses Bayesian prior information about the expected return and uses this information to select new policies for execution. They also emphasize that policy executions generate trajectory data, which can contain more information than only the final return. Their work therefore shows that BO can be adapted to the sequential nature of RL by incorporating trajectory information into the surrogate model.

A related perspective is presented by Letham and Bakshy (2019), who apply Bayesian Optimization for policy search in settings where online experiments are combined with offline simulator evaluations. This is relevant because simulators can provide additional evaluations when direct real-world experiments are expensive or limited. However, such approaches also introduce practical challenges, for example when offline simulators are biased or only approximate the real environment. For this thesis, these studies are mainly relevant because they show that BO can be used to optimize policies or policy-related parameters when the evaluation of a policy is expensive and only observable through experimentation or simulation.

2.6.3 Bayesian Optimization for Automated Reinforcement Learning

A second and more relevant form of interaction is the use of BO for the automated design or configuration of RL systems. RL performance is often sensitive to modelling choices and hyperparameters. These include the learning rate, discount factor, exploration parameters, reward design, state representation, and algorithmic settings. Selecting these components manually can require substantial trial and error. This motivates the use of automated methods that treat the RL configuration process itself as an optimization problem.

Barsce et al. (2018) propose an autonomous reinforcement learning framework that integrates Bayesian Optimization with Gaussian Process regression to optimize RL hyperparameters. In their formulation, the selected hyperparameters are given to the agent, the agent learns a policy, and the resulting performance is returned to the optimizer. The history of evaluated hyperparameter-performance pairs is then used by BO to select the next hyperparameter configuration. This is closely related to the use of BO for Q-learning hyperparameter tuning in this thesis.

The broader field of Automated Reinforcement Learning (AutoRL) extends this idea beyond simple hyperparameter tuning. Afshar et al. (2026) describe AutoRL as a framework in which different components of RL, including MDP modelling, algorithm selection, and hyperparameter optimization, can be automated. This is important because RL applications often require many design decisions before the learning process can begin. Automating these components can reduce the dependency on expert knowledge and make RL more accessible for applied optimization problems.

Several studies illustrate specific forms of such automation. Afshar et al. (2023) propose an automated DRL pipeline for dynamic pricing that includes MDP modelling, algorithm selection, and hyperparameter optimization. Their pipeline uses a hyperparameter optimization method that integrates Bayesian Optimization. Beeks et al. (2022) use Bayesian Optimization to shape the reward function of a DRL agent for a multi-objective online order batching problem. In that case, BO is used to tune the relative importance of different reward components so that the DRL agent can balance multiple objectives. Yoo et al. (2025) use BO in a bi-level framework for MDP state design in DRL-based manufacturing scheduling. In their approach, BO operates at the upper level to select state features, while RL operates at the lower level to evaluate the resulting state representation through agent training.

These examples show that BO can support RL in several ways. It can optimize policy parameters directly. It can tune learning hyperparameters. It can adjust reward-function weights. It can also support the design of state representations. Across these cases, the common principle is

that the RL component is expensive to evaluate, while BO provides a sample-efficient strategy for selecting the next configuration to test.

2.6.4 Positioning of This Thesis

This thesis adopts a complementary BO-RL perspective. BO and RL are not treated as identical alternatives for solving the same subproblem. Instead, they are used for different optimization roles within the same discrete-event simulation case study. BO is used for parametric optimization, while Q-learning is used for control optimization.

The first role of BO is the optimization of static simulation configurations. In this setting, BO searches for station and worker capacity settings before the simulation run. The simulation then evaluates the resulting configuration by producing performance indicators that are combined into a cost function. This corresponds to a black-box simulation optimization problem.

The second role of BO is the tuning of Q-learning hyperparameters. Here, the candidate configuration does not describe the laboratory system directly. Instead, it describes learning-related settings of the RL agent. The performance of each hyperparameter configuration can only be evaluated after the agent has interacted with the simulation and produced a policy. This makes RL hyperparameter tuning another expensive black-box optimization problem.

RL is used for a different purpose. The Q-learning agent does not optimize a fixed simulation parameter vector. Instead, it learns an online dispatching policy at the Sorting station. At each decision point, the agent observes the current system state and selects one of several predefined dispatching rules. The simulation then evolves according to this decision and provides feedback for learning.

In contrast to studies where BO is used directly for policy search or for the automated design of complex DRL pipelines, this thesis uses BO and RL in distinct but connected roles. BO supports sample-efficient search over static configurations and RL hyperparameters. Q-learning supports adaptive decision-making during the simulation run. This separation is methodologically useful because it reflects the difference between parametric optimization and control optimization introduced earlier in this chapter. It also keeps the RL action space interpretable, because the agent selects between predefined dispatching rules rather than learning an opaque end-to-end scheduling policy.

The resulting framework is therefore not a general AutoRL pipeline. It is a DES-based applied optimization study in which BO and RL are combined pragmatically. BO is used where the problem can be expressed as an expensive black-box parameter search. RL is used where decisions must be made repeatedly under changing system states. Together, these methods provide two complementary ways of improving operational performance in the simulated laboratory system.

3 Methodology

This chapter describes the methodological approach used to answer the research questions defined in Section 1.3. The methodology is structured into three main parts: the simulation model and the modifications required for the respective optimization approaches, the experimental design, and the implementation of the optimization algorithms in Python.

3.1 Simulation Model

The base simulation model used in this thesis was developed by the supervisors for a previous module of the ADLS Bachelor's degree program. It represents a discrete-event simulation of a testing laboratory and is implemented in Python using the salabim library (van der Ham, 2023). The model simulates the flow of orders through the laboratory stations and the different analysis steps that must be performed for each order. The following sections describe the base simulation model and the modifications introduced in this thesis to enable simulation optimization using Bayesian Optimization and Reinforcement Learning.

3.1.1 Base Simulation Model

The base simulation model provided by the supervisors represents a testing laboratory and is implemented in the script `lab_analysis_simulation_evalfailure.py`. It is a discrete-event simulation implemented in Python using the salabim library. The model describes the flow of orders through a sequence of laboratory stations and analysis steps. In the context of this thesis, the simulated entities are referred to as orders, although they can be interpreted as laboratory samples or testing requests. The process flow consists of the following stations:

- Preparation
- Sorting
- Analysis 1
- Analysis 2
- Evaluation
- Dispatching

Each station is represented as a resource with a configurable capacity. The processing time at each station is modelled by a triangular distribution, defined by a lower bound, an upper bound, and a most likely value. After preparation, each order enters the sorting station, where a probabilistic decision determines whether the order is processed by Analysis 1, Analysis 2, or both analysis steps. Orders that are assigned to Analysis 1 may additionally require Analysis 2, while orders that are not assigned to Analysis 1 are always processed by Analysis 2.

After the required analysis steps have been completed, the order enters the evaluation station. At this stage, a probabilistic evaluation outcome determines whether the order passes or fails the evaluation. If an order fails the evaluation, it is routed back to the sorting station and must pass through the analysis process again. This retry loop continues until the order successfully passes the evaluation. The complete order flow through the system is illustrated in Figure 1.

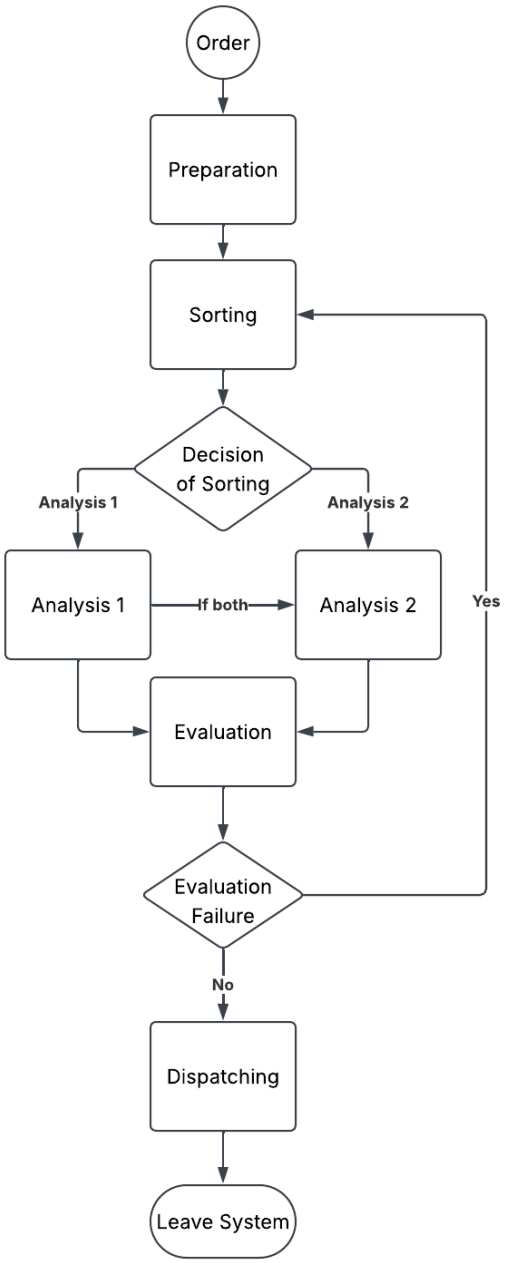


Figure 1: Flowchart of the order process

The analysis stations additionally require workers to perform the corresponding analysis tasks. Therefore, the model distinguishes between station capacities and the shared worker capacity. The relevant configurable capacity parameters of the base simulation are the capacities of the preparation, sorting, analysis, evaluation, and dispatching stations, as well as the available worker capacity. For reproducibility, the main stochastic input parameters of the base simulation model are summarized in Table 1.

Table 1: Main input parameters of the base simulation model

Model component	Distribution or parameter	Value
Simulation horizon	Deterministic duration	1440 min, corresponding to one simulated day; no warm-up period was discarded
Order arrivals	Stationary stochastic arrival mode used in the final BO and RL experiments: $I \sim$ Exponential(mean $= \mu_I$) with $\mu_I = r \cdot 1440/500$ min	The constant 500 denotes the baseline number of orders per simulated day at $r = 1$. In this mode, r scales the mean interarrival time, so smaller values of r generate more orders; the expected number of orders per day is $500/r$. The implementation also supports hourly arrival rates from `hourly_arrival_rates.csv`.
Preparation processing time	Triangular distribution Tri(a, m, b)	Tri$(1.5, 2.0, 2.5)$ min
Sorting processing time	Triangular distribution Tri(a, m, b)	Tri$(3.0, 4.0, 5.0)$ min
Analysis 1 processing time	Triangular distribution Tri(a, m, b)	Tri$(3.0, 4.0, 6.0)$ min
Analysis 2 processing time	Triangular distribution Tri(a, m, b)	Tri$(2.0, 2.0, 2.0)$ min
Evaluation processing time	Triangular distribution Tri(a, m, b)	Tri$(0.2, 0.4, 0.75)$ min
Dispatching processing time	Triangular distribution Tri(a, m, b)	Tri$(0.8, 1.0, 1.2)$ min
Transport duration	Deterministic duration	1 min
Initial analysis assignment	Bernoulli decision	$P(\text{Analysis 1}) = 0.8$
Additional Analysis 2 after Analysis 1	Conditional Bernoulli decision	$P(\text{Analysis 2} \mid \text{Analysis 1}) = 0.5$
Analysis 2 without Analysis 1	Deterministic routing rule	$P(\text{Analysis 2} \mid \neg\text{Analysis 1}) = 1.0$
Evaluation failure	Bernoulli decision	$P(\text{failure}) = 0.1$, $P(\text{pass}) = 0.9$

3.1.2 BO Simulation Model

The simulation model used for Bayesian Optimization is based on the original laboratory simulation implemented in `lab_analysis_simulation_evalfailure.py`. The general process flow and the main station logic remain largely unchanged. However, the model was extended in `lab_analysis_simulation_BO.py` and `base_library.py` to introduce additional system complexity and to make the optimization problem more meaningful. The two main extensions are the introduction of order due dates and the implementation of station input and output buffers.

Due dates:
Each order is assigned an individual due date when it is created. In the BO simulation model, the due date D_i of order i is sampled from a continuous uniform distribution,

$$D_i \sim \mathcal{U}(4 \cdot 60,\ 8 \cdot 60), \tag{9}$$

where D_i is measured in minutes. Equivalently, the due date is uniformly distributed between 4 and 8 hours after order creation, resulting in a mean due date of 6 hours. An order is classified as late if its time in the system exceeds its assigned due date. This extension adds an additional performance dimension to the simulation, since orders should not only be processed in high quantities, but also within a given time frame.

The due date mechanism is particularly relevant for the cost function described in Section 3.2.1, as it allows late orders to be penalized explicitly. Without this extension, an order could remain in the system until all required resources become available, without any direct negative consequence apart from a longer average time in the system. By introducing due dates, the simulation better reflects practical constraints in testing laboratories, where samples or testing requests often have to be completed within a defined time window.

Queue buffers:
A second extension concerns the handling of queues and station buffers. In the original simulation model, orders request the required station resource directly and the station is released immediately after processing is completed. Although salabim internally maintains request queues for occupied resources, this structure does not explicitly model blocking effects between consecutive stations. As a result, downstream congestion has only a limited effect on upstream stations.

To address this limitation, the BO simulation model introduces an input buffer and an output buffer for each station in `base_library.py`. Before an order can enter a station, it must first claim the station's input buffer. After processing, the order claims the station's output buffer before the station resource itself is released. The output buffer is only released once the order can move on to the next station. This mechanism creates a more restrictive and realistic flow of orders through the laboratory, because limited downstream availability can block upstream processing.

These buffer mechanisms make bottlenecks more explicit and increase the relevance of capacity-related decisions. Consequently, the capacities of the preparation, sorting, analysis, evaluation, dispatching, and worker resources become meaningful decision variables for Bayesian Optimization. The optimizer can therefore evaluate how different capacity configurations affect throughput, lateness, work in progress, and the overall cost function.

3.1.3 RL Simulation Model

The Reinforcement Learning simulation model is implemented in the files `lab_analysis_simulation_RL.py` and `base_library_RL.py`. It is based on the same general laboratory process as the base and BO simulation models, including preparation, sorting, analysis, evaluation, and dispatching. Furthermore, the RL model also includes due dates and evaluation failures. However, the internal simulation structure differs substantially from the BO model, because the RL agent must be able to interact with the simulation while it is running. In contrast, the BO model treats the simulation as a black-box evaluation function for complete parameter configurations, as described in Section 3.2.3.

The main purpose of the RL simulation model is to allow an agent to influence dispatching decisions at the sorting station. Therefore, the simulation was reformulated from an order-driven process model into a station-driven model with explicit queues and active station servers. In the base and BO models, each order controls its own movement through the system by requesting station resources sequentially. In the RL model, orders are instead placed into explicit station queues, while separate station server components select and process orders from these queues.

Active station architecture:
The RL model introduces the classes `StationQueue`, `RLStation`, and `RLStationServer`. Each station has an explicit queue and one or more active server components, depending on the configured station capacity. When an order enters a queue, idle servers are activated and may select an order for processing. This architecture makes it possible to intervene at a decision point before an order is selected from a queue.

The capacity of a station is represented by the number of parallel `RLStationServer` components. For example, a sorting capacity of two creates two active sorting servers. The analysis stations additionally request the shared worker resource during processing, as in the other simulation models. Thus, both station capacities and worker capacity remain relevant constraints.

Order flow and routing logic:
Orders are represented by the class `OrderRL`. When an order is created, it receives a starting time, a due date, and additional state variables such as the current queue, the queue entry time, and the number of failed evaluations. After creation, the order enters the preparation queue and is then passivated, meaning that its own process is suspended until it is reactivated by a station server. From this point onward, the movement of the order through the model is controlled by the station servers rather than by the order itself.

After preparation, the order is routed to the sorting queue. At sorting, the simulation determines whether the order requires Analysis 1, Analysis 2, or both, using the same probability-based logic as in the other simulation models. After the required analysis steps, the order enters the

evaluation queue. If the evaluation fails, the order is routed back to the sorting queue as rework. If the evaluation succeeds, the order is sent to dispatching and then leaves the system.

RL decision point:
The RL agent controls only the sorting station. A decision is made when a sorting server is available and at least two orders are waiting in the sorting queue. If fewer than two orders are available, no meaningful dispatching choice exists and the station uses the default FIFO action. The agent does not select a specific order directly. Instead, it selects one of four interpretable dispatching rules:

- FIFO
- Earliest Due Date
- Longest Waiting Time
- Highest Lateness Risk

This design keeps the action space small and interpretable. It also makes the learned policy easier to compare with fixed dispatching-rule baselines. The dispatching rules are described in detail in Section 3.3.3.

State representation.
The RL dispatcher constructs a discrete state representation at each decision point. The state consists of five components:

- the binned length of the sorting queue,
- the binned length of the evaluation queue,
- the binned number of orders currently in the system,
- the fraction of orders close to their due date,
- the fraction of orders that have already failed at least one evaluation.

The continuous and count-based state variables are discretized before they are used as Q-table keys. Table 2 summarizes the binning scheme. An order is considered close to its due date if its remaining slack time, defined as the absolute due date minus the current simulation time, is at most 120 minutes. If no orders are currently in the system, both fraction-based state components are assigned to bin 0.

Table 2: Discretization of the Q-learning state representation

State component	**Raw value**	**Bin**
Sorting queue length	0	0
Sorting queue length	1–3	1
Sorting queue length	4–7	2
Sorting queue length	≥ 8	3
Evaluation queue length	0	0
Evaluation queue length	1–3	1
Evaluation queue length	4–7	2
Evaluation queue length	≥ 8	3
Orders in system	0	0
Orders in system	1–10	1
Orders in system	11–25	2
Orders in system	≥ 26	3
Fraction close to due date	0	0
Fraction close to due date	$(0, 0.25]$	1
Fraction close to due date	> 0.25	2
Fraction with failed evaluation	0	0
Fraction with failed evaluation	$(0, 0.25]$	1
Fraction with failed evaluation	> 0.25	2

This results in a maximum discrete state space of $4 \times 4 \times 4 \times 3 \times 3 = 576$ states. With four available dispatching actions, the corresponding tabular Q-learning representation contains up to

2304 state-action entries, although only the states encountered during training are stored in the implemented Q-table.

These state variables were chosen to provide the agent with information about local congestion at the controlled station, downstream congestion at evaluation, the overall workload in the system, due-date pressure, and the amount of rework.

Reward structure:
The RL model includes an explicit reward signal for training the agent. Rewards are accumulated between decision points and are then used to update the Q-learning agent. Completed orders receive a positive reward. Orders that exceed their due date receive an additional penalty, and evaluation failures are penalized because they create rework. In addition, small penalties are applied for work in progress and total queue length at decision points. This reward structure encourages the agent to process orders efficiently, avoid late completions, and reduce congestion in the system.

Agent types and evaluation:
The simulation supports three agent types. A baseline agent always applies one fixed dispatching rule. A random agent selects among the available dispatching rules uniformly at random. The Q-learning agent uses a tabular Q-learning approach with epsilon-greedy exploration. Each simulation run corresponds to one RL episode, and the agent is updated based on the observed state transitions and accumulated rewards.

Compared with the BO simulation model, the RL simulation model therefore changes the role of the simulation. The BO model is used as a black-box evaluation function for static capacity configurations. The RL model, by contrast, exposes decision points during the simulation run and allows an agent to adapt dispatching decisions dynamically based on the current system state. This makes it possible to investigate whether dynamic rule selection at the sorting station can improve performance compared with fixed dispatching rules or random dispatching decisions.

3.2 BO Methodology

This section describes the implementation of Bayesian Optimization in Python and the experimental design used to answer RQ1 and RQ2, as defined in Section 1.3. RQ1 is addressed through a descriptive evaluation of the configurations and performance values obtained by Bayesian Optimization, whereas H1 and H2 provide hypothesis-based tests for the comparative aspects of RQ2.

To create a meaningful optimization problem, the order-arrival intensity was calibrated empirically to achieve a late-order fraction of approximately 50%. For the BO simulation model, this calibration was performed with the script `rate_multiplier_calibration.py`. The script evaluated candidate values of the implementation parameter `rate_multiplier` from 0.1 to 3.0 in steps of 0.1 and simulated each candidate with ten random seeds. For each candidate, the mean fraction of late completed orders was calculated, and the value closest to the target late-order fraction of 50% was selected. Based on this calibration, the BO experiments used a `rate_multiplier` of 0.5. With the constant-arrival process and a simulation horizon of 1440 minutes, this setting generated approximately 1000 orders per simulated day. This configuration produced a sufficiently constrained system in which late orders occur regularly, making capacity changes relevant for the optimization. Because the calibration was based on simulation outputs, the resulting late-order fraction should be interpreted as an approximate operating point rather than an exact target value.

3.2.1 Experimental Setup

The experimental setup was designed to evaluate RQ1 descriptively and to test the two hypotheses associated with RQ2. The optimization problem is defined as a minimization problem. The decision variables are the capacities of the laboratory stations and the shared worker resource. The following integer-valued parameter ranges were used:

- Preparation capacity: 1 to 5
- Sorting capacity: 1 to 5
- Analysis 1 capacity: 1 to 5

- Analysis 2 capacity: 1 to 5
- Evaluation capacity: 1 to 5
- Dispatching capacity: 1 to 5
- Worker capacity: 1 to 10

Since no real monetary cost data was available, a synthetic cost function was defined. The purpose of this function is to represent a laboratory that aims to process as many orders as possible, avoid late or unfinished orders, and at the same time avoid unnecessarily high resource capacities. The implemented objective function is:

$$\begin{aligned} f(\theta) = & - w_c \cdot \frac{n_{\text{completed}}}{n_{\text{created}}} + w_l \cdot \frac{n_{\text{late}} + n_{\text{incomplete}}}{n_{\text{created}}} \\ & + w_s \cdot \frac{\sum_{s \in \mathcal{S}} c_s - 6}{30 - 6} + w_w \cdot \frac{c_{\text{workers}} - 1}{10 - 1} + w_t \cdot \frac{\overline{T}_{\text{system}}}{T_{\text{due,mean}}} \end{aligned} \tag{10}$$

where θ denotes one capacity configuration and $\mathcal{S}$ is the set of the six station resources: preparation, sorting, Analysis 1, Analysis 2, evaluation, and dispatching. The term $\sum_{s \in \mathcal{S}} c_s$ therefore denotes the total station capacity. Since each of the six station capacities can take values from 1 to 5, the total station capacity is normalized using its minimum value 6 and maximum value 30. Similarly, the worker capacity c_{workers} is normalized using its search range from 1 to 10.

The remaining terms describe the simulated order outcomes. n_{created} is the number of orders created during one simulation run, $n_{\text{completed}}$ is the number of orders completed before the end of the run, and $n_{\text{incomplete}}$ is the number of orders still in the system at the end of the run. n_{late} denotes the subset of completed orders whose time in system exceeds their assigned due date. Thus, late completed orders are still counted as completed in the throughput term, but receive an additional lateness penalty in the second term. This reflects the modelling assumption that completing an order is beneficial, but completing it late is less desirable than completing it within its due date.

$\overline{T}_{\text{system}}$ denotes the mean time in system of completed orders. The mean due date $T_{\text{due,mean}}$ is six hours, corresponding to the midpoint of the due date distribution used in the simulation. The objective is minimized; lower values therefore indicate a more favourable trade-off between throughput, service performance, time in system, and resource usage.

Table 3: Symbols used in the BO objective function

Symbol	Definition
θ	Capacity configuration evaluated by the optimizer
$\mathcal{S}$	Set of six station resources: preparation, sorting, Analysis 1, Analysis 2, evaluation, and dispatching
c_s	Capacity of station s
c_{workers}	Capacity of the shared worker resource
n_{created}	Number of orders created during a simulation run
$n_{\text{completed}}$	Number of orders completed during a simulation run
n_{late}	Number of completed orders whose time in system exceeds their due date
$n_{\text{incomplete}}$	Number of orders still in the system at the end of the simulation run
$\overline{T}_{\text{system}}$	Mean time in system of completed orders
$T_{\text{due,mean}}$	Mean due date, equal to six hours in the BO experiments

For the default experiments, the following weights were used:

- $w_c = 1.0$ for completed orders
- $w_l = 3.0$ for late or incomplete orders
- $w_s = 0.8$ for station capacity
- $w_w = 0.35$ for worker capacity
- $w_t = 0.4$ for mean time in system

These weights place the strongest emphasis on avoiding late and incomplete orders. To investigate RQ2, an additional weight configuration was evaluated in which capacity costs were weighted more strongly and late or incomplete orders were penalized less severely. For this configuration, the weights were changed to $w_c = 0.35$, $w_l = 0.8$, $w_s = 3.0$, and $w_w = 1.0$, while the weight for mean time in system remained $w_t = 0.4$. This allows the experiments to examine whether Bayesian Optimization identifies different capacity configurations when the optimization objective shifts from service performance toward resource efficiency.

Each evaluated parameter configuration was simulated using multiple replications to reduce the effect of stochastic variation. For each trial, 30 simulation replications were executed with different random seeds. The objective value of a trial was calculated as the mean objective value across these replications. For the final comparison, 20 independent BO runs and 20 independent Random Search runs were executed. Each run used 30 evaluated configurations, resulting in the same evaluation budget for both methods.

3.2.2 Statistical Evaluation

The BO and Random Search results were statistically evaluated using the independent multi-run results. No separate statistical hypothesis test was defined for RQ1; it was assessed descriptively using the best objective values, selected capacity configurations, and simulation KPIs obtained by BO. For H1, the final best objective values of the 20 independent BO runs were compared with the final best objective values of the 20 independent Random Search runs. Because the number of independent runs is limited and no normality assumption was made, the two groups were compared using a two-sided Mann-Whitney U test at a significance level of $\alpha = 0.05$.

For H2, the objective values of the default and changed weight settings were not compared directly, because changing the weights changes the scale and interpretation of the objective function. Instead, H2 was evaluated by comparing the best capacity configurations identified by BO under the two weight settings. The primary comparison was based on the total selected capacity of the best configuration from each independent BO run. These total capacities were compared using a two-sided Mann-Whitney U test at $\alpha = 0.05$. In addition, the individual station and worker capacities were summarized descriptively and tested exploratorily with Holm-Bonferroni correction.

For all statistical comparisons, mean, standard error, median, and interquartile range were reported in addition to the test statistic and p-value.

3.2.3 Implementation

Bayesian Optimization was implemented using the `ax-platform` library, an open-source optimization framework provided by Meta ("Ax", n.d.). The implementation uses the `AxClient` interface to define the experiment, specify the integer-valued search space, and iteratively request new parameter configurations. The objective is defined as a minimization objective.

The optimization loop works as follows. First, `AxClient` proposes a capacity configuration within the defined parameter bounds. This configuration is passed to `lab_analysis_simulation_BO.py`, where the simulation is executed for 30 replications. For each replication, the relevant KPIs are recorded, including completed orders, late orders, incomplete orders, time in system, work in progress, and the evaluated capacities. The objective value is then calculated using the cost function implemented in `objective_utils.py`. After all replications of a trial have been completed, the mean objective value and its standard error are passed back to `AxClient`. Based on the previously observed configurations and objective values, Ax updates its optimization model and proposes the next configuration. This process is repeated until the trial budget is exhausted.

Random Search was implemented as a baseline using the same simulation model, search space, objective function, number of trials, and number of replications. Instead of using previous observations to guide the search, Random Search samples each capacity configuration uniformly from the defined parameter ranges. This makes it possible to compare whether Bayesian Optimization finds better configurations than unguided search under the same evaluation budget.

Single-run scripts were created to test both Bayesian Optimization and Random Search on a local machine. However, the full experimental setup is computationally demanding because each method

requires multiple independent runs, each consisting of many simulation replications. Therefore, the multi-run experiments were executed on the ZHAW HPC infrastructure using SLURM job arrays. The resulting trial and replication files were then combined using `combine_multi_run_results.py`. The statistical analysis of the combined BO and Random Search results was performed with `statistical_analysis_BO.py`.

3.3 RL Methodology

This section describes the experimental setup and implementation used to evaluate RQ3 and RQ4, as defined in Section 1.3. The RL experiments investigate whether a Q-learning agent can improve system performance through dynamic dispatching decisions at the sorting station compared with fixed dispatching rules and a random dispatching agent. In addition, Bayesian Optimization is used to tune selected Q-learning hyperparameters.

As in the BO experiments described in Section 3.2, the RL simulation was calibrated empirically to create a constrained system with a late-order fraction of approximately 50%. However, the calibrated `rate_multiplier` from the BO model was not transferred directly to the RL model, because the two simulation variants differ in their internal dynamics. The BO model includes buffer-based blocking effects and evaluates static capacity configurations, whereas the RL model uses explicit station queues, station-driven server components, and fixed station and worker capacities of one. As a result, the same arrival-process parameter does not necessarily lead to the same congestion level or late-order fraction in both models.

Therefore, the script `rate_multiplier_calibration_RL.py` was used to evaluate different arrival-rate multipliers under the RL baseline configuration. Based on this calibration, a `rate_multiplier` of 0.9 was selected, as it produced a late-order fraction close to the target value of 50%. With the same 1440-minute simulation horizon, this setting generated approximately 558 orders per simulated day in the holdout evaluation. All station capacities and the worker capacity were kept fixed at one in the RL experiments to focus the comparison on dispatching decisions rather than capacity optimization.

3.3.1 Experimental Setup

The experimental setup was designed to evaluate RQ3 and RQ4 and their corresponding hypotheses H3 and H4. The comparison includes fixed dispatching-rule baselines, a random dispatching agent, and a Q-learning agent.

Four fixed baseline agents were evaluated. Each baseline always applies one dispatching rule at the sorting station: FIFO, Earliest Due Date, Longest Waiting Time, or Highest Lateness Risk. These baselines make it possible to assess whether the Q-learning agent learns a useful dynamic switching policy rather than merely reproducing one fixed rule. A random agent was also included as an additional reference point. It selects one of the available dispatching rules uniformly at random whenever a decision is required.

For the manually configured Q-learning experiments, the agent was trained for 10,000 episodes. After training, the learned policy was evaluated using common evaluation seeds shared with the fixed baselines and the random agent to ensure comparability. The default Q-learning configuration used a learning rate $\alpha = 0.1$, a discount factor $\gamma = 0.95$, an initial exploration rate $\epsilon = 1.0$, an epsilon decay of 0.9995 for the 10,000-episode run, and a minimum epsilon of 0.05. This default configuration is referred to as `manual standard` in the results and discussion and uses the original lateness-risk setting with $t_1 = 0$ and a risk window of 120 minutes. In addition, two manually configured variants were evaluated with the same Q-learning hyperparameters but different fixed lateness-risk settings: `manual fixed120` and `manual fixed758`. These variants use the fixed120 and fixed758 lateness-risk settings summarized in Table 4. Final policy comparisons were based on the independent holdout evaluation described below.

The reward signal was defined directly in the simulation and accumulated between two consecutive RL decision points. A completed order receives a positive reward of 10. If an order is completed after its due date, an additional penalty of 20 is applied, resulting in a net reward of -10 for a late completed order. Evaluation failures receive a penalty of 3 because they create rework and send the order back to the sorting station. In addition, congestion-related penalties

are applied at each decision point. Specifically, the accumulated transition reward is reduced by 0.001 for each order currently in the system and by 0.002 for each order waiting in any station queue.

The transition reward passed to the Q-learning agent can therefore be written as

$$r_t = 10 \cdot N_{\text{completed},t} - 20 \cdot N_{\text{late},t} - 3 \cdot N_{\text{evalfail},t} - 0.001 \cdot WIP_t - 0.002 \cdot Q_{\text{total},t}, \tag{11}$$

where $N_{\text{completed},t}$, $N_{\text{late},t}$, and $N_{\text{evalfail},t}$ denote the number of completed orders, late completed orders, and evaluation failures accumulated since the previous decision point. WIP_t denotes the number of orders currently in the system at decision point t, and $Q_{\text{total},t}$ is the total number of orders waiting across all explicit station queues. This reward structure encourages the agent to complete orders, avoid late completions, reduce rework, and limit congestion.

For RQ4, Bayesian Optimization was used for hyperparameter tuning. The tuning objective was to maximize the mean total reward of the trained Q-learning agent. In the implementation, Ax minimizes the negative mean reward, i.e. −mean(total_reward).

The hyperparameter tuning was conducted in several variants. In an initial tuning run, the Q-learning hyperparameters and the parameters of the Highest Lateness Risk dispatching rule were optimized jointly. This joint search included the learning rate α, the discount factor γ, the final exploration rate $\epsilon_{\min}$, the lateness-risk threshold t_1, and the lateness-risk window. In the tuning implementation, $\epsilon_{\min}$ corresponds to the configured target final exploration rate; the epsilon decay was then derived from this value and the number of training episodes. Based on this initial run, the lateness-risk threshold was fixed at approximately $t_1 = 175$ minutes for subsequent experiments. The risk-window parameter did not show a single clearly dominant setting. Therefore, two fixed lateness-risk variants were evaluated: `fixed120`, using a risk window of 120 minutes, and `fixed758`, using a risk window of approximately 758 minutes. In both variants, only the Q-learning hyperparameters α, γ, and $\epsilon_{\min}$ were optimized, while the lateness-risk parameters were kept fixed.

Because the selected discount factor γ varied strongly across the tuning runs, two additional variants were evaluated under the fixed758 lateness-risk setting. The `gamma_low` variant restricted the discount factor to a low range, $\gamma \in [0.0, 0.2]$, while the `gamma_high` variant restricted it to a high range, $\gamma \in [0.8, 0.99]$. The exact fixed lateness-risk values used for the fixed758 setting were $t_1 = 175.214912$ minutes and a risk window of 758.004 minutes; throughout the results, this setting is referred to as `fixed758`. Both `gamma_low` and `gamma_high` used this same fixed758 lateness-risk setting and optimized only α, γ, and $\epsilon_{\min}$. The initial joint search is referred to as `joint100` in the results, because it used 100 BO trials and jointly optimized both the Q-learning hyperparameters and the lateness-risk parameters.

The notation `fixed120`, `fixed758`, `gamma_low`, `gamma_high`, and `joint100` is used throughout the results and discussion to refer to these tuning variants. In both cases, the second threshold of the soft lateness-risk function is defined as $t_2 = t_1 + \text{risk\_window}$. Thus, `fixed120` corresponds to $t_1 \approx 175$ and $t_2 \approx 295$ minutes, while `fixed758` corresponds to $t_1 \approx 175$ and $t_2 \approx 933$ minutes. The main staged tuning setup consisted of 100 BO trials.

Table 4: Search spaces of the evaluated Q-learning tuning variants

Variant	**Lateness-risk setting**	$\boldsymbol{\alpha}$	$\boldsymbol{\gamma}$	$\boldsymbol{\epsilon_{\min}}$	**Tuned risk parameters**
`joint100`	Jointly optimized	[0.005, 0.7]	[0.2, 0.95]	[0.02, 0.22]	$t_1 \in [40, 360]$, risk window $\in [120, 1800]$
`fixed120`	$t_1 \approx 175$, risk window = 120	[0.0001, 0.08]	[0.01, 0.95]	[0.0005, 0.08]	None
`fixed758`	$t_1 \approx 175$, risk window ≈ 758	[0.0001, 0.08]	[0.01, 0.95]	[0.0005, 0.08]	None
`gamma_low`	$t_1 \approx 175$, risk window ≈ 758	[0.0001, 0.08]	[0.0, 0.2]	[0.0005, 0.08]	None
`gamma_high`	$t_1 \approx 175$, risk window ≈ 758	[0.0001, 0.08]	[0.8, 0.99]	[0.0005, 0.08]	None

In Stage 1, each trial trained a Q-learning agent for 1000 episodes and evaluated it on 10 evaluation seeds. In Stage 2, the best Stage-1 candidates were retrained for 10,000 episodes and evaluated on

30 evaluation seeds. Shorter preliminary runs with 50 BO trials and 100 training episodes were used only to explore suitable parameter bounds before the full HPC experiments were executed. These preliminary runs were not used for the final comparison because they used a smaller evaluation budget.

After the Stage-2 tuning comparison, an independent holdout evaluation was conducted to obtain less selection-biased performance estimates for the final RL comparisons. The holdout evaluation used 100 new evaluation seeds from 900000 to 999000 in steps of 1000, and all evaluated policies were run on the same holdout seeds. The evaluated policies comprised the four fixed dispatching-rule baselines, the random agent, three manually configured Q-learning agents, and the five BO-tuned Stage-2 candidates. Although all manually configured Q-learning variants were included in the holdout evaluation for completeness, the hypothesis-based policy comparisons focus on the manual `fixed758` configuration. This was done because `fixed758` was selected as the final BO-tuned variant; using the same lateness-risk setting for the manually configured Q-learning agent and the Highest Lateness Risk baseline keeps the policy comparison aligned with the final tuned configuration. For Q-learning policies, the stored Q-tables from the corresponding training runs were loaded and re-evaluated without additional training. Thus, Stage 2 was used as the model-selection procedure, while the holdout evaluation was used for the final performance estimates and hypothesis-based policy comparisons.

3.3.2 Statistical Evaluation

The RL results were evaluated using common random seeds, so policy comparisons were treated as paired comparisons. The primary performance measure was the total reward per evaluation run. For the final RL results, the independent 100-seed holdout evaluation was used.

For H3, the manually configured fixed758 Q-learning agent was compared separately against FIFO, Earliest Due Date, Longest Waiting Time, Highest Lateness Risk, and the random agent using two-sided Wilcoxon signed-rank tests at a significance level of $\alpha = 0.05$. Because H3 involves multiple reference-policy comparisons, the resulting p-values were adjusted using the Holm-Bonferroni method.

For H4, the selected BO-tuned fixed758 Q-learning configuration from the Stage-2 tuning procedure was compared with the manually configured fixed758 Q-learning configuration on the same 100 holdout seeds. This comparison was also performed using a two-sided Wilcoxon signed-rank test on total reward.

Additional paired Wilcoxon signed-rank tests were used for direct final-policy comparisons, including the comparison between the BO-tuned fixed758 policy and the Highest Lateness Risk baseline. Where multiple reference comparisons were reported for the same selected policy, Holm-Bonferroni-adjusted p-values were used. For all RL comparisons, mean, standard error, median, and interquartile range were reported together with the test statistic and p-value.

3.3.3 Implementation

The RL implementation is contained in `lab_analysis_simulation_RL.py`, `RL_agents.py`, `RL_experiment.py`, and the hyperparameter tuning scripts in the `RL` directory. The RL agent controls only the sorting station. A decision point occurs when a sorting server is available and at least two orders are waiting in the sorting queue. If fewer than two orders are available, the model uses FIFO because no meaningful dispatching choice exists.
The action space consists of four dispatching rules:

- **FIFO**: selects the order that entered the sorting queue first.
- **Earliest Due Date**: selects the order with the earliest absolute due date.
- **Longest Waiting Time**: selects the order with the longest current waiting time in the sorting queue.
- **Highest Lateness Risk**: selects the order with the highest calculated lateness-risk score. If no order is close to its due date, the rule falls back to the longest-waiting order to avoid starvation.

The action space consists of four interpretable dispatching rules. The agent does not select an individual order directly, but selects one rule whenever a decision point occurs. The selected rule is then applied to the current sorting queue.

- **FIFO**: selects the order that entered the sorting queue first. This rule is based only on the queue entry time and therefore represents a simple first-come-first-served policy.
- **Earliest Due Date**: selects the order with the earliest absolute due date, calculated as the order creation time plus its assigned due-date allowance. In contrast to FIFO, this rule ignores the order's queue entry time except as a tie-breaker and prioritizes orders according to due-date urgency.
- **Longest Waiting Time**: selects the order with the longest current waiting time in the sorting queue. In the implemented single-queue setting, this rule is closely related to FIFO because waiting time is determined by the queue entry time. It is nevertheless included as an explicit rule because it represents a waiting-time-based dispatching principle.
- **Highest Lateness Risk**: calculates a lateness-risk score for each order based on its remaining slack time and selects the order with the highest risk. If multiple orders have the same risk score, the order with the longer waiting time is selected. If no order has a positive lateness-risk score, the rule falls back to the longest-waiting order to avoid starvation.

For the Highest Lateness Risk rule, the slack time of order i at time t is defined as

$$s_i(t) = d_i - t, \tag{12}$$

where d_i is the absolute due date of the order. The lateness-risk score is calculated using a soft threshold function:

$$\rho_i(t) = \begin{cases} 1, & s_i(t) \le t_1, \\ \dfrac{t_2 - s_i(t)}{t_2 - t_1}, & t_1 < s_i(t) < t_2, \\ 0, & s_i(t) \ge t_2. \end{cases} \tag{13}$$

Orders with slack below or equal to t_1 are therefore treated as fully at risk, while orders with slack above or equal to t_2 are treated as not at risk. Between t_1 and t_2, the risk score increases linearly as the remaining slack decreases. This mechanism is illustrated in Figure 2 using the fixed758 setting, which was selected for the final RL policy comparisons. In this setting, the lower threshold is approximately $t_1 = 175$ minutes and the upper threshold is approximately $t_2 = 933$ minutes.

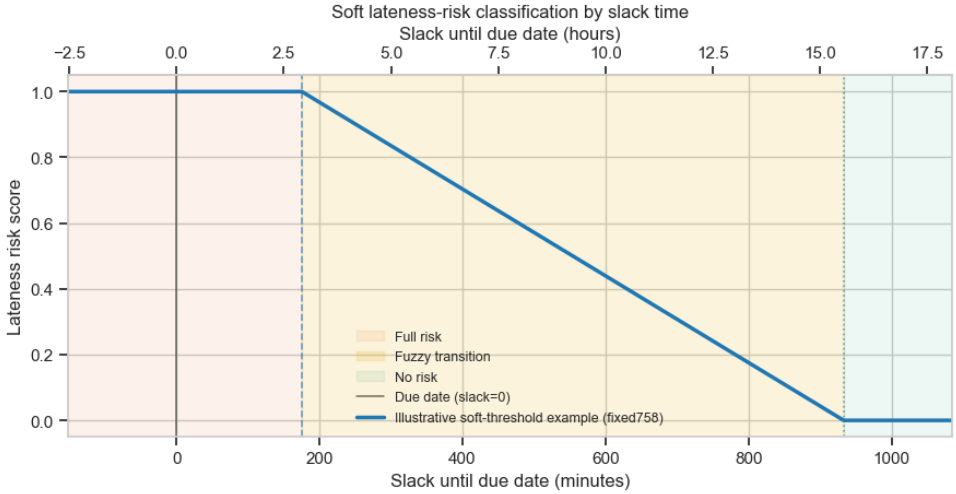


Figure 2: Illustrative soft threshold function used to calculate the lateness-risk score, shown for the fixed758 setting.

The state representation is constructed by the `RLDispatcher`. The discretization of these components is defined in Table 2. It contains five discrete components: the binned sorting queue length, the binned evaluation queue length, the binned number of orders currently in the system, the

fraction of orders close to their due date, and the fraction of orders that have already failed at least one evaluation. This state representation provides the agent with information about local congestion, downstream congestion, overall workload, due-date pressure, and rework.

The Q-learning agent is implemented in `RL_agents.py`. It uses a tabular Q-table and epsilon-greedy action selection. During training, the agent updates the Q-table whenever a new decision point is reached. Rewards are accumulated between two decision points and then passed to the agent together with the previous state, the selected action, and the current state. At the end of each simulation episode, a final update is performed.

The standard experiment script `RL_experiment.py` executes the baseline comparisons, random-agent evaluations, Q-learning training, and final Q-learning evaluation. The results are written to separate CSV files for baselines, random-agent runs, Q-learning training, and Q-learning evaluation. The trained Q-table is saved as a pickle file for later analysis.

The independent holdout evaluation was executed with `rl_holdout_evaluation.py`. `run_RL_holdout_evaluation.sh` runs the holdout policies as a SLURM array on the ZHAW HPC, after which the Python script combines the per-policy CSV files. The notebook `rl_holdout_evaluation_analysis.ipynb` then computes the holdout descriptives, paired Wilcoxon tests with Holm-Bonferroni corrections, and exports the corresponding result tables and figures.

Hyperparameter tuning was implemented with Ax in a staged workflow. The script `rl_stage1_bo.py` performs the initial Bayesian Optimization over shorter training runs. Each trial calls `run_single_rl_bo_trial.py`, trains a Q-learning agent, evaluates it, and returns the negative mean reward to Ax. The best configurations are then passed to Stage 2, where `run_single_rl_stage2_candidate.py` retrains selected candidates for 10,000 episodes and evaluates them with 30 seeds. The scripts `run_RL_stage1_BO.sh` and `run_RL_stage2_array.sh` were used to execute these experiments on the ZHAW HPC infrastructure.

4 Results

This chapter presents the empirical results of the Bayesian Optimization and Reinforcement Learning experiments described in Section 3. The results are structured according to the research questions and hypotheses introduced in Section 1.3.

The chapter focuses on observed outcomes, descriptive statistics, and statistical test results. Interpretation of the findings, possible explanations, and implications for simulation optimization are discussed separately in Section 5.

4.1 Bayesian Optimization Results

This section presents the Bayesian Optimization results. It first summarizes the optimization outcomes under the default objective weights, then compares Bayesian Optimization with Random Search, and finally evaluates how changing the objective weights affected the selected capacity configurations.

4.1.1 Optimization Performance and Selected Capacity Configurations

The Bayesian Optimization results under the default objective weights were evaluated across 20 independent optimization runs. Each run consisted of 30 evaluated capacity configurations, and each configuration was simulated using 30 replications. This subsection reports the final best objective values, the selected capacity configurations, and the resulting operational simulation KPIs.

Figure 3 shows the development of the best objective value found so far over the 30 optimization trials. The line represents the median across the 20 independent BO runs, while the shaded area shows the interquartile range. The median best objective value decreased over the optimization budget, indicating that later trials generally improved upon the configurations found in earlier trials.

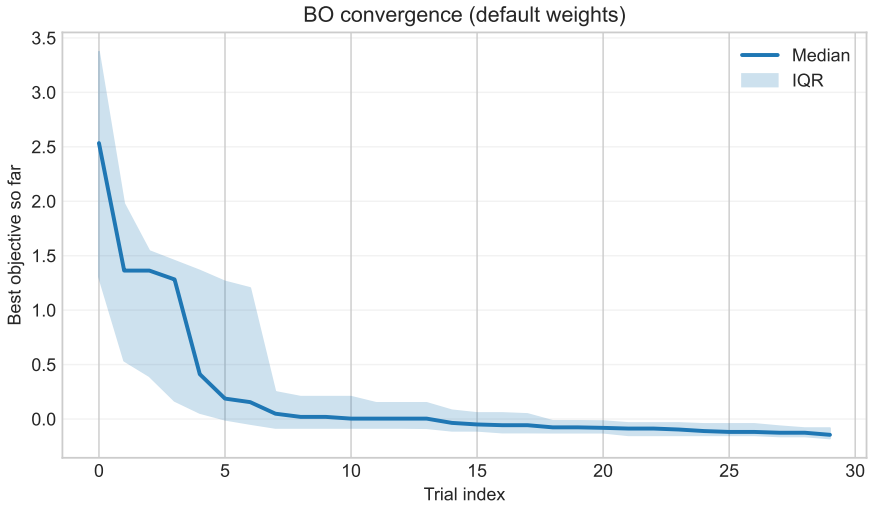


Figure 3: Median best objective value found so far across 20 Bayesian Optimization runs under the default objective weights. The shaded area represents the interquartile range.

Across the 20 independent runs, the final best objective value had a mean of -0.1341 with a standard error of 0.0180. The median final best objective value was -0.1450, with an interquartile range from -0.1784 to -0.0838. The best observed final objective value across all BO runs was -0.2700, while the highest final best objective value was 0.0350.

Table 5 summarizes the capacity configurations selected as the best configuration in each BO run. The selected configurations used comparatively high capacities for sorting, Analysis 1, evaluation, and workers. Dispatching capacity was consistently low, with a median value of one.

Table 5: Summary of best capacity configurations selected by Bayesian Optimization under the default objective weights across 20 independent runs

Capacity	Mean	SE	Median	Q1	Q3	Min–Max
Preparation	3.20	0.21	3.00	2.00	4.00	2–5
Sorting	4.30	0.13	4.00	4.00	5.00	3–5
Analysis 1	4.20	0.14	4.00	4.00	5.00	3–5
Analysis 2	3.55	0.26	3.50	3.00	4.25	1–5
Evaluation	4.55	0.14	5.00	4.00	5.00	3–5
Dispatching	1.45	0.15	1.00	1.00	2.00	1–3
Workers	6.90	0.35	7.00	6.00	7.50	5–10
Total capacity	28.15	0.56	28.50	26.00	30.00	24–32

The distribution of selected capacity configurations across the individual BO runs is shown in Figure 4. The figure displays the best capacity configuration of each independent run and illustrates the variation in selected capacities across runs.

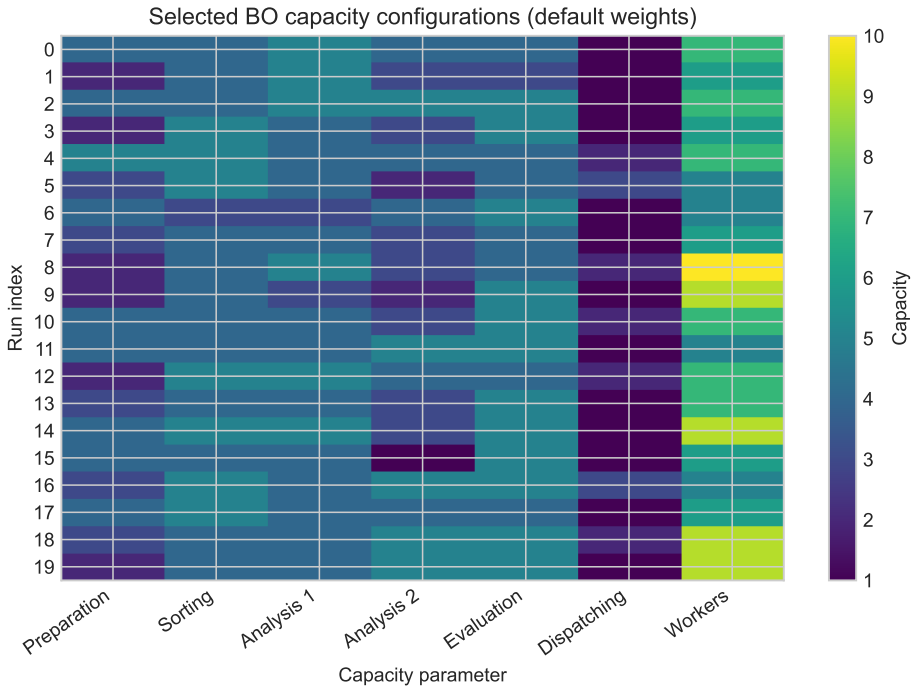


Figure 4: Best capacity configurations selected by Bayesian Optimization under the default objective weights across 20 independent runs.

Table 6 reports the operational KPIs obtained from the best BO configuration of each run. On average, the selected configurations created approximately 998 orders and completed approximately 940 orders during the simulated time horizon. The mean number of incomplete orders was 58.2. No late completed orders were observed in the best configurations selected by BO under the default objective weights.

Table 6: Operational KPIs of the best Bayesian Optimization configurations under the default objective weights across 20 independent runs

Metric	Mean	SE	Median	Q1	Q3
Created orders	997.94	1.51	998.55	994.12	1001.42
Completed orders	939.73	9.77	948.33	914.89	968.86
Incomplete orders	58.22	9.52	50.22	26.58	80.28
Late completed order fraction	0.000	0.000	0.000	0.000	0.000
Mean time in system (min)	36.68	5.05	23.58	23.13	42.49
Mean WIP	33.84	4.19	28.52	17.88	46.77

4.1.2 Comparison Between Bayesian Optimization and Random Search

Bayesian Optimization was compared with Random Search using the final best objective value obtained in each independent run. Both methods were evaluated with the matched optimization budget defined in Section 3.2.1. Lower objective values indicate better configurations. The comparison was conducted separately for the default objective weights and the changed objective weights.

Figure 5 shows the distribution of final best objective values for Bayesian Optimization and Random Search under both objective-weight settings. Under the default objective weights, Bayesian Optimization achieved lower final best objective values on average than Random Search. Under the default objective weights, Bayesian Optimization achieved lower objective values descriptively, but the difference was just above the 5% significance threshold and was therefore not treated as statistically significant ($p = 0.0531$).

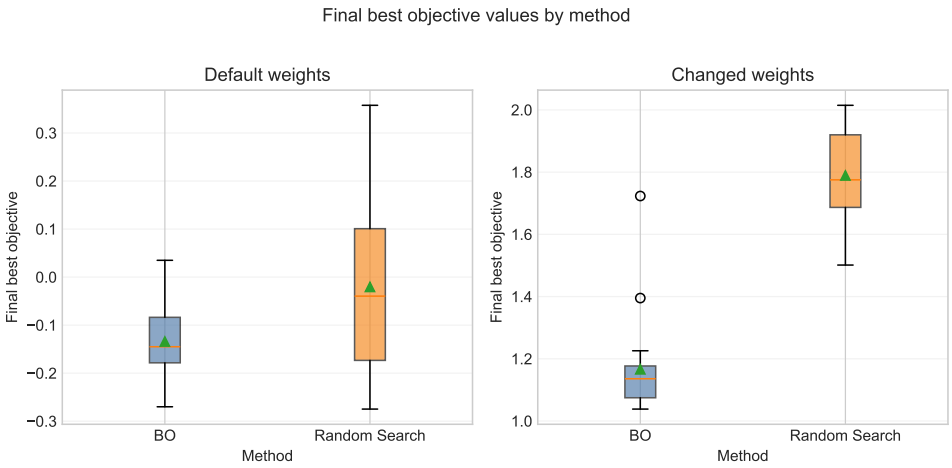


Figure 5: Final best objective values obtained by Bayesian Optimization and Random Search under the default and changed objective weights across 20 independent runs.

The descriptive statistics are reported in Table 7. For the default objective weights, Bayesian Optimization reached a mean final best objective value of −0.1341, compared with −0.0204 for Random Search. For the changed objective weights, Bayesian Optimization reached a mean final best objective value of 1.1662, compared with 1.7893 for Random Search.

Table 7: Final best objective values obtained by Bayesian Optimization and Random Search

Weights	Method	n	Mean	SE	Median	IQR
Default	BO	20	-0.1341	0.0180	-0.1450	0.0947
Default	Random Search	20	-0.0204	0.0415	-0.0396	0.2742
Changed	BO	20	1.1662	0.0347	1.1359	0.1021
Changed	Random Search	20	1.7893	0.0324	1.7752	0.2332

The statistical test results are shown in Table 8. For the default objective weights, the Mann-Whitney U test did not indicate a statistically significant difference between Bayesian Optimization and Random Search at $\alpha = 0.05$. For the changed objective weights, Bayesian Optimization achieved significantly lower final best objective values than Random Search.

Table 8: Mann-Whitney U test results for Bayesian Optimization and Random Search

Comparison	Mean difference	U statistic	p-value	Significant
Default weights	-0.1136	128.0	0.0531	No
Changed weights	-0.6231	8.0	2.22×10^{-7}	Yes

These results provide partial support for H1. Bayesian Optimization achieved significantly lower final best objective values than Random Search under the changed objective weights. Under the default objective weights, Bayesian Optimization achieved lower objective values descriptively, but the difference was not statistically significant at the 5% level.

4.1.3 Effect of Objective Weighting on Selected Capacities

Changing the objective weights changes the scale and interpretation of the objective function. Therefore, the two objective-weight settings were not compared using their objective values. Instead, the comparison focuses on the capacity configurations selected by Bayesian Optimization. The primary comparison for H2 was based on the total selected capacity of the best BO configuration in each independent run.

Figure 6 shows the median selected capacities under the default and changed objective weights. The changed objective weights led to lower selected capacities across all capacity parameters. The strongest absolute differences were observed for the total capacity, the worker capacity, and the evaluation capacity.

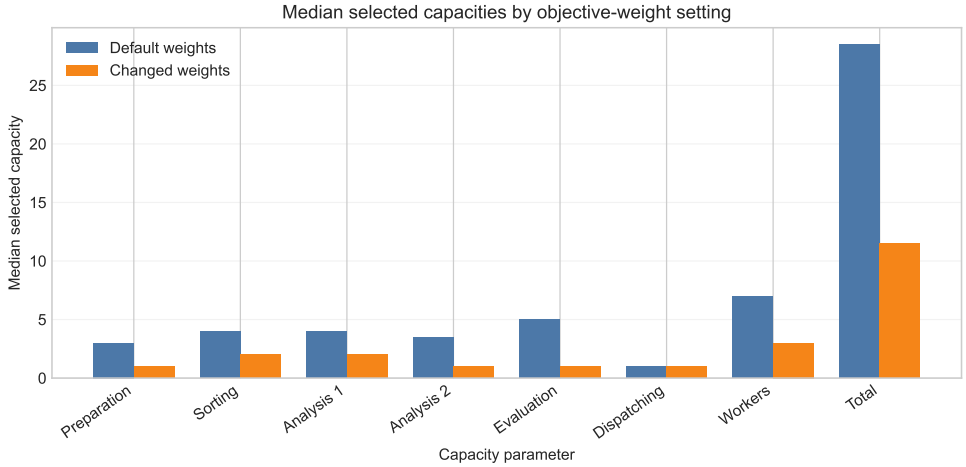


Figure 6: Median selected capacities of the best Bayesian Optimization configurations under the default and changed objective weights across 20 independent runs.

Under the default weights, the mean total selected capacity was 28.15, with a median of 28.5. Under the changed weights, the mean total selected capacity decreased to 11.30, with a median of 11.5. The Mann-Whitney U test indicated a statistically significant difference in total selected capacity between the two objective-weight settings ($U = 400.0$, $p = 6.18 \times 10^{-8}$).

Table 9 summarizes the individual selected capacity components under both weight settings. The primary total-capacity comparison is reported separately in the text above with its unadjusted p-value. The individual station and worker capacities were compared exploratorily using Mann-Whitney U tests with Holm-Bonferroni correction. All component-wise comparisons remained statistically significant after correction.

Table 9: Individual selected capacity components under default and changed objective weights

Capacity	Default median	Changed median	Default mean	Changed mean	Adj. p-value
Preparation	3.0	1.0	3.20	1.20	4.27×10^{-7}
Sorting	4.0	2.0	4.30	2.20	4.27×10^{-7}
Analysis 1	4.0	2.0	4.20	1.95	4.03×10^{-7}
Analysis 2	3.5	1.0	3.55	1.10	4.03×10^{-7}
Evaluation	5.0	1.0	4.55	1.30	1.30×10^{-7}
Dispatching	1.0	1.0	1.45	1.05	0.0188
Workers	7.0	3.0	6.90	2.50	3.18×10^{-7}

These results support H2. Changing the relative weights of order-related and capacity-related cost components led Bayesian Optimization to select substantially different capacity configurations. In particular, the changed objective weights resulted in significantly lower total selected capacities.

4.2 Reinforcement Learning Results

This section presents the results of the Reinforcement Learning experiments. First, the Bayesian Optimization-based hyperparameter-tuning variants are compared to select the final tuned Q-learning configuration. The following sections then compare the Q-learning agent with fixed

dispatching-rule baselines and evaluate the effect of Bayesian Optimization-based hyperparameter tuning.

4.2.1 Comparison of Hyperparameter-Tuning Variants

Several Bayesian Optimization-based hyperparameter-tuning variants were evaluated for the Q-learning agent. Using the staged tuning procedure described in Section 3.3.1, the comparison in this subsection is based on the best Stage-2 candidate of each tuning variant. The primary selection criterion was the mean total reward in Stage-2 evaluation.

Figure 7 shows the Stage-2 reward distributions of the best candidates from the evaluated tuning variants. The `fixed758` variant achieved the highest mean total reward among the evaluated candidates. The differences between `fixed758`, `gamma_high`, `gamma_low`, and `joint100` were relatively small in terms of mean reward.

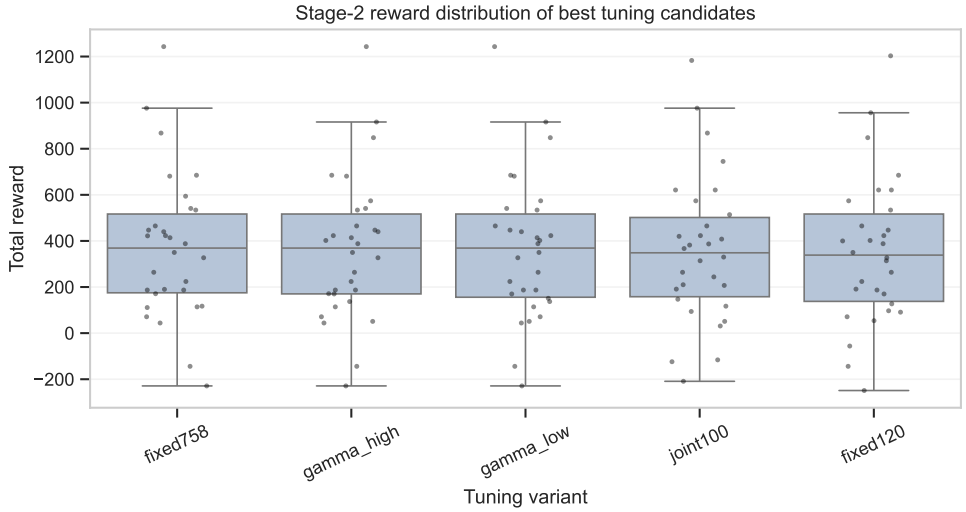


Figure 7: Stage-2 total reward distributions of the best candidates from the evaluated Q-learning hyperparameter-tuning variants.

Table 10 summarizes the selected hyperparameters and Stage-2 evaluation results. The `fixed758` variant achieved the highest mean total reward of 370.17 and also the lowest mean late order fraction among the evaluated variants. The Stage-2 comparison was used as the selection procedure for the final BO-tuned configuration.

Table 10: Best Stage-2 candidates of the evaluated Q-learning hyperparameter-tuning variants

Variant	α	γ	$\epsilon_{\min}$	Risk window	Mean reward	Median reward	Mean late frac.
`fixed758`	0.0089	0.9500	0.0031	758.0	370.17	369.0	0.4183
`gamma_high`	0.0057	0.8130	0.0005	758.0	364.17	369.0	0.4194
`gamma_low`	0.0039	0.2000	0.0442	758.0	363.50	369.0	0.4195
`joint100`	0.0050	0.2000	0.0200	120.0	356.83	348.5	0.4206
`fixed120`	0.0162	0.9500	0.0005	120.0	352.83	338.5	0.4214

To avoid using the same evaluation replications both for selecting and for assessing the final policy, the subsequent hypothesis-based comparisons are based on the independent 100-seed holdout evaluation.

On the holdout seeds, `fixed758` also achieved the highest mean total reward among the BO-tuned variants, although the differences among the best variants were small. Table 11 summarizes the holdout results of the BO-tuned variants. The `fixed758` variant obtained a mean reward of 223.19 and the lowest mean late order fraction among the BO-tuned variants at 0.4442. It was followed closely in mean reward by `gamma_low` with 221.59 and `gamma_high` with 218.79. However, the median rewards of `gamma_low` and `gamma_high` were slightly higher than that of `fixed758`. Therefore, `fixed758` should be interpreted as the Stage-2-selected variant that also achieved the highest mean holdout reward among the BO-tuned variants, not as a uniformly dominant configuration.

Table 11: Holdout descriptive results for the BO-tuned Q-learning variants

Variant	n	Mean reward	SE	Median reward	Mean late frac.
`fixed758`	100	223.19	30.69	237.0	0.4442
`gamma_low`	100	221.59	31.01	237.5	0.4444
`gamma_high`	100	218.79	30.97	237.5	0.4449
`fixed120`	100	210.99	29.94	192.5	0.4463
`joint100`	100	207.99	30.24	206.5	0.4468

Figure 8 shows the corresponding holdout reward distributions. The distributions overlap substantially, which reinforces that the holdout differences between the best BO-tuned variants should be interpreted cautiously.

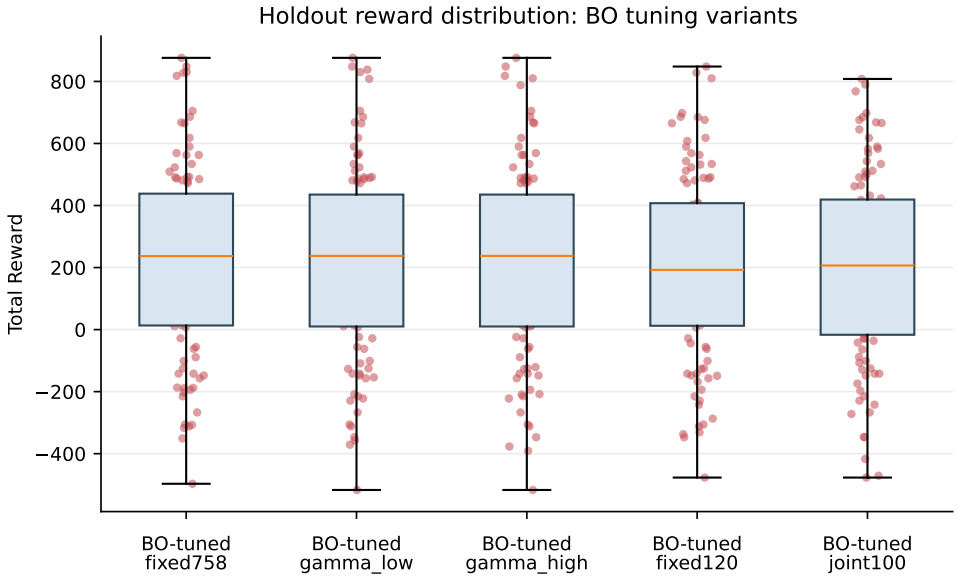


Figure 8: Holdout total reward distributions of the selected BO-tuned Q-learning variants across 100 independent evaluation seeds.

4.2.2 Manual Q-learning Compared with Dispatching Baselines

Based on the tuning-variant comparison in the previous subsection, the fixed758 lateness-risk setting defined in Section 3.3.1 was used for the following manual Q-learning policy comparison. The other manually configured Q-learning variants were therefore not used for the hypothesis-based comparison in this subsection. The manually configured fixed758 Q-learning agent was compared with the random agent and four fixed dispatching-rule baselines: FIFO, Earliest Due Date, Longest Waiting Time, and Highest Lateness Risk. In the implemented single-queue setting at the sorting station, FIFO and Longest Waiting Time are behaviorally equivalent: the order that entered the queue first is also the order with the longest waiting time. They are nevertheless reported separately because they represent distinct dispatching rules in the implementation.

The comparison followed the paired holdout evaluation and Holm-Bonferroni correction procedure described in Section 3.3.2.

Table 12 reports the descriptive holdout statistics for total reward and the mean late order fraction. The manually configured fixed758 Q-learning agent achieved a mean total reward of 208.59. This was higher than FIFO, Longest Waiting Time, Earliest Due Date, and the random agent, but lower than the Highest Lateness Risk baseline. The same pattern is visible in the mean late order fraction, where Highest Lateness Risk achieved the lowest value among the policies in this comparison.

Table 12: Holdout descriptive results for manual fixed758 Q-learning and dispatching baselines

Policy	n	Mean reward	SE	Median	IQR	Mean late frac.
Highest Lateness Risk	100	223.19	29.93	257.0	394.25	0.4442
Manual Q-learning	100	208.59	30.28	215.5	413.50	0.4467
FIFO	100	-12.21	29.34	-2.5	400.50	0.4860
Longest Waiting Time	100	-12.21	29.34	-2.5	400.50	0.4860
Random agent	100	-26.01	29.42	2.5	408.50	0.4885
Earliest Due Date	100	-108.61	29.34	-77.5	410.50	0.5033

The statistical comparison results are shown in Table 13. Manual fixed758 Q-learning achieved significantly higher total rewards than the random agent, Earliest Due Date, FIFO, and Longest Waiting Time after Holm-Bonferroni correction. However, it achieved significantly lower total rewards than the Highest Lateness Risk baseline.

Table 13: Paired Wilcoxon signed-rank tests comparing manual fixed758 Q-learning with reference policies on holdout seeds

Comparison	Mean diff.	Median diff.	Adj. p-value	Significant
Manual Q-learning vs Random agent	234.60	240.0	1.80×10^{-17}	Yes
Manual Q-learning vs Earliest Due Date	317.20	320.0	1.80×10^{-17}	Yes
Manual Q-learning vs FIFO	220.80	220.0	1.80×10^{-17}	Yes
Manual Q-learning vs Longest Waiting Time	220.80	220.0	1.80×10^{-17}	Yes
Manual Q-learning vs Highest Lateness Risk	-14.60	-20.0	0.0044	Yes

The identical results for FIFO and Longest Waiting Time follow from their equivalence in the single-queue setting and are therefore expected. These results provide partial support for H3. The manually configured fixed758 Q-learning agent outperformed the random agent and three of the four fixed dispatching-rule baselines, but it did not outperform the Highest Lateness Risk baseline. Instead, Highest Lateness Risk performed significantly better than manual fixed758 Q-learning on the holdout seeds.

4.2.3 Effect of Bayesian Optimization-Based Hyperparameter Tuning

The effect of Bayesian Optimization-based hyperparameter tuning was evaluated on the independent holdout seeds by comparing the BO-tuned fixed758 Q-learning agent with the manually configured fixed758 Q-learning agent. The comparison followed the paired holdout testing procedure described in Section 3.3.2.

Figure 9 shows the holdout total reward distributions of the two Q-learning configurations. The BO-tuned fixed758 agent achieved a higher mean total reward than the manually configured fixed758 agent, although the difference was much smaller than in the Stage-2 evaluation.

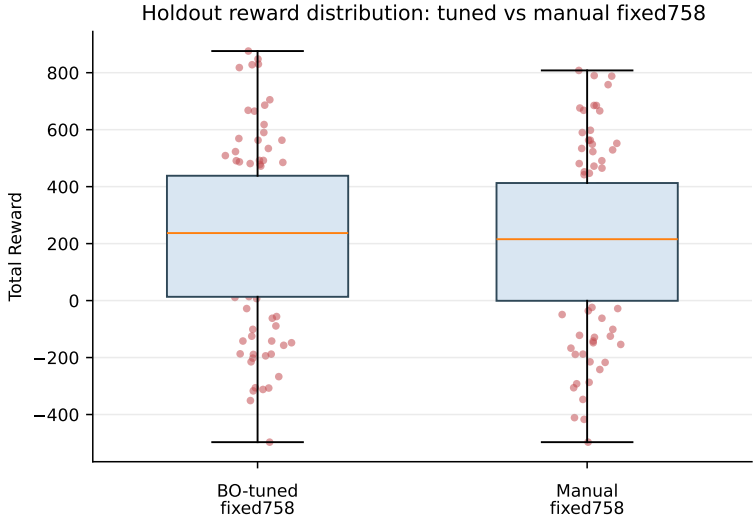


Figure 9: Holdout total reward distributions of BO-tuned and manually configured Q-learning under the fixed758 setting.

The descriptive holdout results for both configurations are included in the final policy summary in Table 14. The BO-tuned fixed758 agent achieved a mean total reward of 223.19, compared with 208.59 for the manually configured fixed758 agent. The mean late order fraction was 0.4442 for the BO-tuned agent and 0.4467 for the manually configured agent.

The paired Wilcoxon signed-rank test indicated a statistically significant difference between the two Q-learning configurations. The BO-tuned fixed758 agent achieved a mean reward difference of 14.60 compared with the manually configured fixed758 agent, with a median paired difference of 0.0 ($W = 423.0$, $p = 0.000405$). Thus, the improvement is statistically detectable on the paired holdout seeds, but its practical magnitude is small, especially given the median paired difference of zero.

These results support H4, but the estimated improvement is substantially smaller on the independent holdout seeds than in the Stage-2 evaluation.

4.2.4 Final Tuned Policy Performance

The final tuned policy selected for the RL evaluation was the BO-tuned fixed758 Q-learning agent. The previous subsection showed that this configuration significantly improved Q-learning performance compared with the manually configured fixed758 agent on the independent holdout seeds. To summarize the final RL results, Figure 10 and Table 14 compare the final tuned policy with the manually configured Q-learning agent, the fixed dispatching-rule baselines, and the random agent.

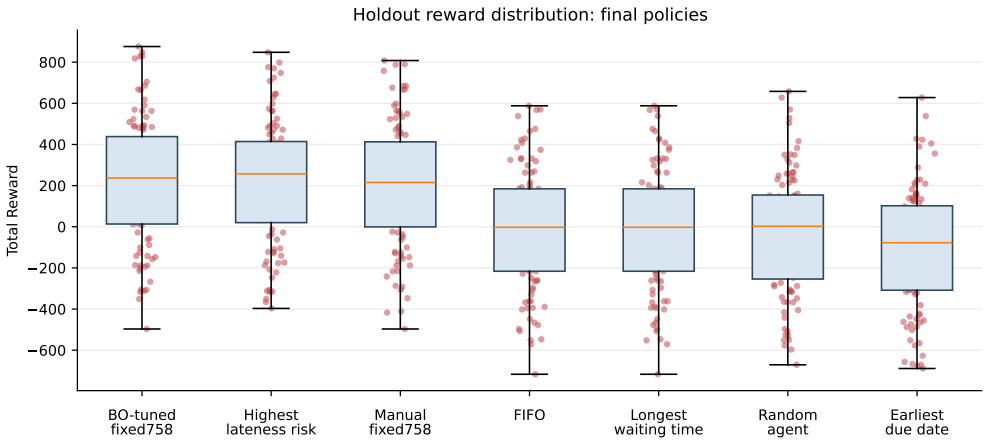


Figure 10: Holdout total reward distributions of the final BO-tuned fixed758 policy, reference policies, and the random agent.

Table 14: Final holdout comparison of the BO-tuned fixed758 policy and reference policies

Policy	Mean reward	Median reward	IQR	Mean late frac.
BO-tuned Q-learning	223.19	237.0	425.00	0.4442
Highest Lateness Risk	223.19	257.0	394.25	0.4442
Manual Q-learning	208.59	215.5	413.50	0.4467
FIFO	-12.21	-2.5	400.50	0.4860
Longest Waiting Time	-12.21	-2.5	400.50	0.4860
Random agent	-26.01	2.5	408.50	0.4885
Earliest Due Date	-108.61	-77.5	410.50	0.5033

In the holdout evaluation, the BO-tuned fixed758 Q-learning agent achieved a mean total reward of 223.19, which was identical to the mean reward of the Highest Lateness Risk baseline. Highest Lateness Risk achieved a higher median reward of 257.0 compared with 237.0 for the BO-tuned agent, while both policies had nearly identical mean late order fractions after rounding. The unrounded mean late order fractions differ only marginally (0.444153 for BO-tuned Q-learning and 0.444184 for Highest Lateness Risk), and the identical mean reward results from aggregation across the 100 holdout seeds rather than from identical seed-level outcomes. The paired Wilcoxon signed-rank test found no statistically significant difference between BO-tuned fixed758 and Highest Lateness Risk ($p = 0.8860$).

This result is important for interpreting H3. While the manually configured fixed758 Q-learning agent partially supports H3 by outperforming the random agent and several simple dispatching baselines, neither the manual nor the final BO-tuned Q-learning policy demonstrates a statistically significant advantage over the strongest baseline, Highest Lateness Risk. Therefore, the RL results should be interpreted as evidence that Q-learning can reach a performance level close to a strong domain-specific dispatching heuristic, but not as evidence that it clearly surpasses this heuristic.

4.3 Summary of Hypothesis Results

Table 15 summarizes the results of the hypothesis-based evaluations reported in the previous sections. The table reports only the primary hypothesis-related comparisons; descriptive and exploratory results were reported in the corresponding subsections.

Table 15: Summary of hypothesis-based results

Hyp.	**Primary comparison**	**Test**	**p-value**	**Result**
H1	BO vs. Random Search based on final best objective values. BO was significant under changed weights, but not under default weights.	Mann-Whitney U	0.0531 / 2.22×10^{-7}	Partially supported
H2	Total selected capacity under default vs. changed objective weights.	Mann-Whitney U	6.18×10^{-8}	Supported
H3	Manual fixed758 Q-learning vs. random agent and fixed dispatching-rule baselines. Manual Q-learning outperformed Random, FIFO, Earliest Due Date, and Longest Waiting Time, but Highest Lateness Risk performed significantly better. The final BO-tuned fixed758 policy also did not significantly outperform Highest Lateness Risk.	Wilcoxon signed-rank with Holm correction	see Table 13; final BO-tuned vs. HLR: $p = 0.8860$	Partially supported
H4	BO-tuned fixed758 Q-learning vs. manually configured fixed758 Q-learning on the independent holdout seeds.	Wilcoxon signed-rank	$p = 0.000405$	Supported, small practical effect

Overall, the results show that Bayesian Optimization contributed to both optimization tasks: it identified significantly different capacity configurations under changed objective weights and significantly improved the Q-learning policy compared with the manually configured fixed758 agent. The final BO-tuned fixed758 policy achieved the highest mean holdout reward together with the Highest Lateness Risk baseline, but it did not significantly outperform this strongest dispatching-rule baseline.

5 Discussion

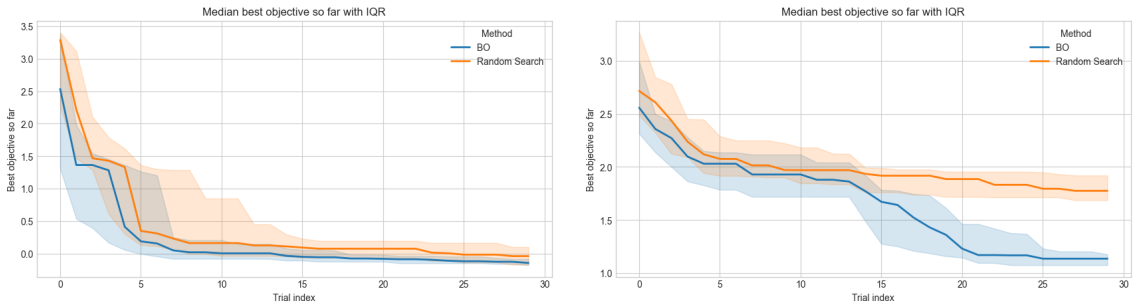
The results show that Bayesian Optimization was effective for the static capacity-configuration problem and also contributed to improving the Q-learning agent through hyperparameter tuning. However, the strength and interpretation of these effects differ between the two optimization settings. While the BO results provide clear evidence that the selected capacity configurations depend strongly on the objective weighting, the evidence for BO outperforming Random Search is stronger under the changed objective weights than under the default weights. For the RL experiments, the tuned Q-learning policy improved over the corresponding manually configured fixed758 Q-learning agent, but it did not clearly outperform the strongest dispatching-rule baseline, Highest Lateness Risk. The following sections discuss these findings in terms of optimization performance, robustness, policy behaviour, and methodological limitations.

5.1 Interpretation of the Bayesian Optimization Results

The Bayesian Optimization results indicate that BO is well suited for the static capacity-configuration problem considered in this thesis. Under the default objective weights, BO selected configurations with relatively high capacities, especially for sorting, analysis, evaluation, and worker resources. This is consistent with the objective formulation, because the default weights strongly reward completed orders and penalize late or incomplete orders. The resulting configurations therefore represent service-oriented solutions rather than resource-minimal designs. Importantly, these configurations should not be interpreted as globally optimal laboratory designs, but as high-performing configurations within the evaluated search budget and under the specific objective function used in the experiment.

The comparison with Random Search further shows that the observed advantage of BO depends on the structure and weighting of the objective function. Under the default weights, BO achieved lower final best objective values descriptively, but the difference was not statistically significant at the 5% level. H1 is therefore only partially supported: BO significantly outperformed Random Search under the changed objective weights, but not under the default objective weights ($p = 0.0531$). A likely explanation is that the default objective makes good solutions comparatively easier to find: configurations with sufficiently high capacities can avoid late completed orders and achieve strong service performance, even if they are not resource efficient. In such a setting, Random Search can occasionally identify competitive high-capacity configurations within the same evaluation budget.

This interpretation is supported by the convergence behaviour shown in Figure 11. Under the default objective weights, both BO and Random Search improve rapidly during the early trials, and the gap between the methods remains relatively small. Under the changed objective weights, however, the difference becomes more pronounced. Random Search plateaus at a higher objective value, whereas BO continues to improve over later trials. This suggests that BO becomes more advantageous when the objective function requires a more difficult trade-off between maintaining acceptable service performance and reducing selected capacities.



Default objective weights | Changed objective weights

Figure 11: Median best objective value found so far by Bayesian Optimization and Random Search under default and changed objective weights. The shaded areas represent the interquartile range across independent runs.

The changed objective weights therefore do not merely alter the numerical scale of the objective function, but change the meaning of what constitutes a good configuration. Under the default weights, high service performance can be achieved by selecting comparatively high capacities.

Under the changed weights, capacity-related costs become more important, and BO identifies substantially lower-capacity configurations. The strong reduction in total selected capacity supports the interpretation that the optimization procedure reacts systematically to the objective formulation rather than simply converging toward one generally best capacity setting.

These findings highlight an important methodological implication for simulation optimization. The performance of an optimizer cannot be assessed independently of the objective function it is optimizing. In this thesis, BO performed particularly well when the objective function encoded a clear trade-off between service performance and resource efficiency. At the same time, the results also show that the selected configurations are only meaningful relative to the chosen weights. Consequently, the BO results should be interpreted as evidence that Bayesian Optimization can efficiently search the static simulation-configuration space, but not as evidence for a universally optimal laboratory configuration.

5.2 Discussion of Reinforcement Learning Results

The Reinforcement Learning results show that Q-learning can learn meaningful dispatching behaviour in the simulated laboratory system. The agent performed substantially better than the random agent and several simple fixed dispatching rules, while the strongest domain-specific heuristic remained a challenging benchmark. The following sections discuss this finding in terms of H3, hyperparameter tuning, stochasticity, and policy behaviour.

5.2.1 Interpretation of Results

The manually configured fixed758 Q-learning agent outperformed the random agent, FIFO, Earliest Due Date, and Longest Waiting Time on the holdout seeds. This indicates that the agent did not merely learn arbitrary action choices, but was able to exploit information from the state representation to select dispatching rules that improved the total reward. In this sense, the RL approach successfully learned a policy that was more effective than several generic or simple dispatching strategies.

However, the comparison with the Highest Lateness Risk baseline limits the strength of this conclusion. Highest Lateness Risk performed significantly better than the manually configured fixed758 Q-learning agent and was not significantly outperformed by the final BO-tuned fixed758 Q-learning policy. The final tuned policy achieved the same mean reward as Highest Lateness Risk in the fixed758 comparison, but Highest Lateness Risk achieved a higher median reward; the identical mean reward results from aggregation across the 100 holdout seeds, while the seed-level outcomes and action-usage patterns differed. This suggests that the learned policy reached a performance level close to the strongest heuristic, but did not robustly surpass it.

A likely explanation is that Highest Lateness Risk already encodes strong domain knowledge about the most relevant operational pressure in the RL task. The calibrated RL simulation creates a congested system in which late orders are common, and the reward function penalizes lateness and evaluation failures. Under these conditions, a heuristic that explicitly prioritizes orders according to lateness risk is well aligned with the objective of the agent. In contrast, the Q-learning agent must infer a useful dispatching strategy from the discretized state representation and delayed reward feedback.

The design of the RL action space is also important for interpreting the results. The agent does not choose individual orders directly. Instead, it selects among four predefined dispatching rules. This makes the learned policy interpretable and keeps the tabular Q-learning problem tractable, but it also constrains the possible improvement over the fixed-rule baselines. If one of the available dispatching rules already performs very well across many system states, the additional benefit of dynamically switching between rules is limited. The RL agent can therefore be competitive without necessarily exceeding the best individual heuristic.

Overall, the RL results provide partial support for H3. Q-learning improved performance relative to the random agent and several simple dispatching-rule baselines, which shows that learning-based dispatching is feasible in the implemented simulation environment. At the same time, neither the manually configured nor the BO-tuned Q-learning policy clearly outperformed Highest Lateness Risk. The main interpretation is therefore that Q-learning can approximate or combine useful

dispatching behaviour, but that a strong domain-specific heuristic remains difficult to beat in this experimental setup. The following sections discuss how hyperparameter tuning, stochasticity, and policy behaviour help explain this result.

5.2.2 Effect of Hyperparameter Tuning

The effect of hyperparameter tuning was evaluated most directly by comparing the BO-tuned fixed758 Q-learning agent with the manually configured fixed758 Q-learning agent. This comparison is important because both policies use the same lateness-risk setting, which keeps the underlying action definition comparable and isolates the effect of the tuned Q-learning hyperparameters more clearly. Therefore, the H4 result should be interpreted as evidence for the benefit of BO-based tuning within the fixed758 Q-learning setup, not as a comparison between all manually configured and tuned RL variants.

The holdout results show that BO-based hyperparameter tuning improved the Q-learning agent relative to the corresponding manual fixed758 configuration. The BO-tuned fixed758 agent achieved a mean reward of 223.19, compared with 208.59 for the manually configured fixed758 agent. It also achieved a slightly lower mean late order fraction of 0.4442 compared with 0.4467. The paired Wilcoxon signed-rank test indicated a statistically significant difference between the two configurations ($p = 0.000405$), which supports H4.

However, the practical magnitude of this improvement should be interpreted cautiously. The mean reward difference was 14.60, while the median paired difference was 0.0. This indicates that the improvement was detectable across the paired holdout seeds, but not consistently large for the typical evaluation seed. The reward distributions in Figure 9 also overlap substantially, which reinforces that the tuning effect was incremental rather than transformative.

Taken together, BO-based hyperparameter tuning improved Q-learning relative to the corresponding manual configuration, but the improvement remained incremental. This suggests that the main performance limitation was not only the choice of Q-learning hyperparameters, but also the structure of the state representation, the restricted heuristic action space, the reward design, and the stochastic simulation environment.

The comparison among BO-tuned variants provides an additional indication that the selected fixed758 configuration should not be interpreted as uniquely dominant. The fixed758 variant was selected in Stage 2 and also achieved the highest mean holdout reward among the BO-tuned variants, but the differences to `gamma_low` and `gamma_high` were small. The overlapping holdout reward distributions in Figure 8 suggest that several tuned configurations reached similar performance levels. Consequently, fixed758 is best interpreted as a methodologically justified selected configuration rather than as a robustly superior hyperparameter setting. The implications of this overlap and the role of stochastic variation are discussed further in the next section.

5.2.3 Robustness, Stochasticity, and Selection Effects

The staged tuning procedure was useful for reducing the hyperparameter search space, but the results also show that Stage 1, Stage 2, and the final holdout evaluation serve different methodological purposes. Stage 1 should be interpreted as a screening step that identifies promising configurations under a limited training and evaluation budget. Stage 2 then provides a more detailed re-evaluation of selected candidates after longer training. However, the Stage-1 ranking did not consistently translate into the Stage-2 ranking, which indicates that short training runs and limited evaluation replications are not sufficient for reliable final policy selection.

This behaviour is visible in Figure 12. The Stage-1 reward estimates are often relatively close within each tuning variant, whereas the corresponding Stage-2 reward estimates spread over a much wider range. This shows that candidates that appear similarly promising during Stage 1 can perform very differently after retraining and re-evaluation in Stage 2. Therefore, Stage 1 was appropriate as a filtering mechanism, but not as a final performance estimate.

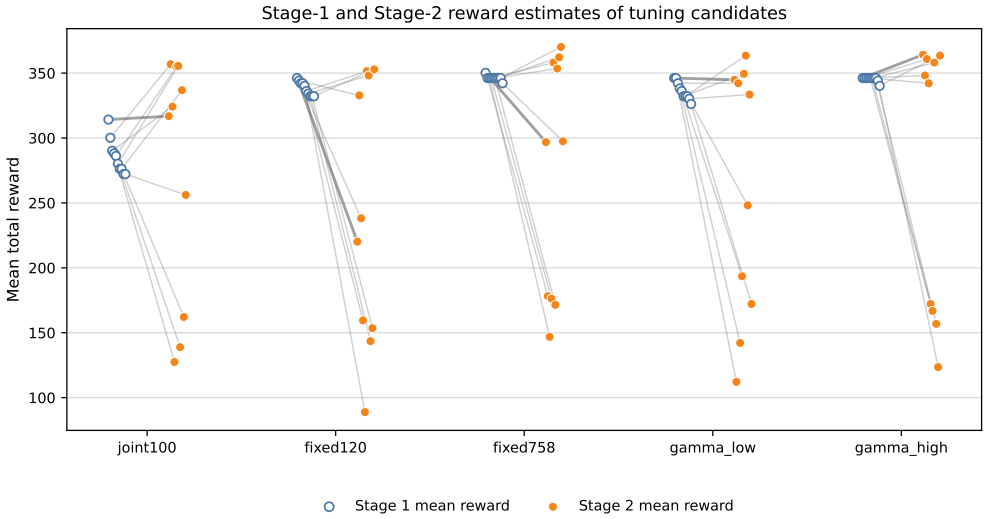


Figure 12: Stage-1 and Stage-2 mean reward estimates of the tuning candidates passed to Stage 2 across the evaluated tuning variants.

The need for an independent holdout evaluation follows directly from this selection process. Because Stage 2 was used to select the final BO-tuned configuration, using the same Stage-2 evaluation seeds as the final performance estimate would risk selection bias. The holdout evaluation therefore provides a less selection-biased estimate of how the selected policies perform on independent simulation seeds. This is particularly important because the Stage-2 mean rewards were substantially higher than the corresponding holdout mean rewards, as summarized in Table 16.

Table 16: Stage-2 and holdout mean rewards of the selected BO-tuned Q-learning variants

Variant	Stage-2 mean reward	Holdout mean reward	Difference
`fixed758`	370.17	223.19	-146.98
`gamma_high`	364.17	218.79	-145.38
`gamma_low`	363.50	221.59	-141.91
`joint100`	356.83	207.99	-148.84
`fixed120`	352.83	210.99	-141.84

These differences should not be interpreted as the policies becoming worse after Stage 2. Rather, they show that Stage-2 performance estimates were influenced by the specific training and evaluation seeds used during the selection procedure. The independent holdout seeds provide a more conservative estimate of final policy performance. This also explains why the advantage of the selected fixed758 configuration appears much smaller in the holdout evaluation than in the Stage-2 comparison.

The training curves further illustrate the stochastic nature of the RL problem. As shown in Figure 13, both the manually configured fixed758 agent and the BO-tuned fixed758 agent show large episode-to-episode variation during training. The rolling mean rewards and best-so-far rewards improve over time, but individual episode rewards remain highly volatile. This indicates that learning progress is not smooth or monotonic, and that isolated high-reward episodes should not be interpreted as stable policy performance.

Figure 13: Training reward over episodes for the manually configured fixed758 Q-learning agent and the BO-tuned fixed758 Q-learning agent. Solid lines show rolling mean rewards, while dashed lines show the best reward observed so far.

The same stochasticity is also visible in the final holdout distributions. The reward distributions of the best BO-tuned variants overlap substantially, and the final BO-tuned fixed758 policy does not clearly separate from the Highest Lateness Risk baseline. This means that small differences in mean reward between the best tuned configurations should be interpreted cautiously. Because the reward estimates have standard errors of around 30 in the holdout comparisons, the second decimal place of the reported mean rewards should not be interpreted as practically meaningful. In particular, the similar holdout performance of `fixed758`, `gamma_low`, and `gamma_high` suggests that the experiments do not identify a uniquely robust optimum for the discount factor or lateness-risk setting.

Overall, the holdout evaluation supports the main qualitative RL findings while tempering the strength of the Stage-2 results. The broad reward distributions, volatile training rewards, and differences between Stage-2 and holdout estimates show that small performance differences should not be overinterpreted. The RL results are therefore best understood as evidence for a competitive but stochastic learning-based dispatching approach.

5.2.4 Policy Behaviour and Action Usage

The previous sections interpreted the RL results mainly through reward values, late-order fractions, and statistical comparisons. The action-usage analysis adds a behavioural perspective by showing how the different Q-learning policies used the available dispatching rules during the holdout evaluation. This analysis is descriptive and is not used as an additional hypothesis test or as a ranking of policy performance. The manual standard and manual fixed120 policies are included only as behavioural context, because they illustrate how different manually selected lateness-risk settings can lead to different learned action preferences. They are not used to revise the hypothesis-based fixed758 comparisons reported in the results chapter.

Figure 14 shows the action usage of the manually configured and BO-tuned Q-learning policies across the 100 holdout seeds. The corresponding percentage values are reported in Table 17. The shares are calculated from the total action counts across all holdout replications of each policy.

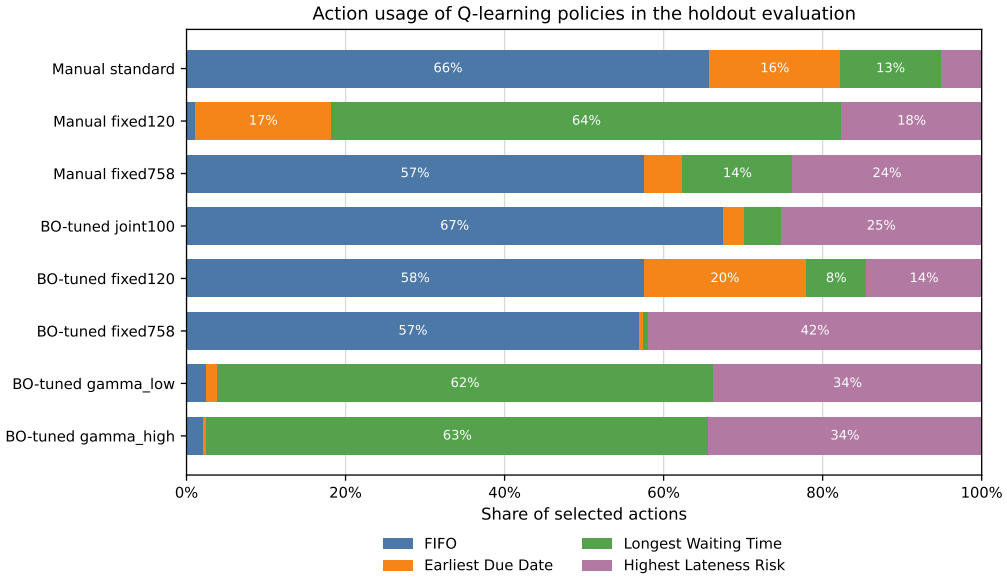


Figure 14: Action usage of manually configured and BO-tuned Q-learning policies in the 100-seed holdout evaluation. Shares are calculated from the total action counts across all holdout seeds.

Table 17: Action usage of Q-learning policies in the holdout evaluation

Policy	**FIFO**	**EDD**	**LWT**	**HLR**
Manual standard	65.8	16.4	12.7	5.1
Manual fixed120	1.1	17.1	64.2	17.6
Manual fixed758	57.5	4.9	13.8	23.8
BO-tuned joint100	67.4	2.7	4.6	25.2
BO-tuned fixed120	57.5	20.4	7.6	14.5
BO-tuned fixed758	57.0	0.4	0.7	41.9
BO-tuned gamma low	2.5	1.3	62.4	33.7
BO-tuned gamma high	2.1	0.4	63.1	34.4

The action shares must be interpreted in light of the implemented queue structure. In the single-queue setting at the sorting station, FIFO and Longest Waiting Time are behaviorally equivalent, because the order that entered the queue first is also the order with the longest waiting time. They are still shown separately because they are separate actions in the Q-learning action space. However, differences between FIFO and Longest Waiting Time should not be interpreted as operationally distinct dispatching behaviour in this model. Instead, their combined share can be understood as the extent to which a policy relies on queue-order-based dispatching rather than due-date-based or lateness-risk-based prioritization.

The comparison between the manually configured fixed758 policy and the BO-tuned fixed758 policy shows that hyperparameter tuning also changed the learned action preferences. Both policies selected FIFO in approximately 57% of their decisions. However, the BO-tuned fixed758 policy selected Highest Lateness Risk in 41.9% of decisions, compared with 23.8% for the manually configured fixed758 policy. At the same time, the BO-tuned fixed758 policy used Earliest Due Date and Longest Waiting Time only rarely. This pattern is consistent with the strong performance of the Highest Lateness Risk baseline, but it does not by itself prove that the increased use of this action caused the reward improvement.

The BO-tuned variants also show that similar holdout rewards can arise from different action mixes. The BO-tuned fixed758 policy relied mainly on FIFO and Highest Lateness Risk, whereas the `gamma_low` and `gamma_high` variants relied mainly on Longest Waiting Time and Highest Lateness Risk. Because FIFO and Longest Waiting Time are equivalent in the implemented single-queue setting, part of this difference reflects a preference between two operationally similar actions. Nevertheless, the results indicate that the tuned Q-learning policies did not converge to one identical action distribution. Instead, several different learned action patterns reached similar performance levels on the holdout seeds.

The manually configured standard and fixed120 policies provide additional behavioural context. The manual standard policy relied mostly on FIFO, while the manual fixed120 policy relied mostly on Longest Waiting Time. This illustrates that the manually selected lateness-risk setting can strongly affect the learned action preferences. However, these policies are included here only to contextualize the action-space behaviour and are not used to change the hypothesis-based interpretation of the fixed758 results.

Overall, the action-usage analysis supports the interpretation that Q-learning learned different but partly equivalent dispatching behaviours. It also shows that BO tuning shifted the fixed758 policy toward a stronger use of Highest Lateness Risk. At the same time, the analysis is aggregated across all states and holdout seeds. It therefore does not show in which specific system states an action was selected. For this reason, the action shares should be interpreted as behavioural context rather than as direct causal evidence for the observed reward differences.

5.3 Implications for Simulation Optimization

The results indicate that simulation-optimization methods should be selected according to the type of decision problem represented by the simulation. Bayesian Optimization was well suited for the static capacity-configuration problem because each candidate solution could be evaluated as a complete simulation configuration. This allowed the simulation model to be treated as a black-box objective function, where each evaluation produced a scalar performance value. For this type of problem, BO provides a structured alternative to unguided search because it can use previous evaluations to guide the selection of new configurations.

At the same time, the BO results show that the optimizer cannot be interpreted independently of the objective function. The selected configurations reflected the relative weighting of service performance and capacity costs. This implies that objective-function design is not merely a technical implementation detail, but a central modelling decision in simulation optimization. In practical applications, the usefulness of an optimized configuration therefore depends on whether the objective function adequately represents the operational priorities of the decision maker.

The RL experiments represent a different type of simulation-optimization problem. Instead of selecting a complete configuration before the simulation starts, the agent makes repeated dispatching decisions while the simulation is running. This makes RL suitable for dynamic decision problems, but also makes the optimization problem more dependent on the state representation, action space, reward design, and benchmark policies. The results suggest that these modelling choices can be at least as important as the choice of hyperparameters. Specific limitations and possible extensions of the RL formulation are discussed in Section 5.4.

The broader implication is that BO and RL are useful for different types of simulation-based decision problems. BO is particularly appropriate when the decision problem can be represented as a static configuration search with expensive but complete simulation evaluations. RL becomes relevant when decisions must be made repeatedly during the simulation and the value of an action depends on the evolving system state. Method selection should therefore be guided less by which algorithm is generally stronger and more by whether the operational problem is primarily parametric, sequential, or a combination of both.

Finally, the experiments highlight the importance of robust evaluation protocols in stochastic simulation optimization. Because simulation outputs vary across random seeds, optimization results should not be assessed only from the same replications that were used for selection. Independent holdout evaluations, common random seeds, distributional summaries, and paired statistical comparisons improve the reliability of the conclusions. This is especially important when performance differences are small or when several configurations achieve similar mean outcomes.

5.4 Limitations and Future Work

5.4.1 Simulation Model and External Validity

The results of this thesis should be interpreted within the scope of the implemented simulation models. Although the BO and RL models represent the same general laboratory process, both

models simplify real operational conditions. For example, arrival patterns, processing-time distributions, due-date generation, evaluation failures, and resource behaviour were defined through model assumptions rather than calibrated from complete real-world production data. In addition, the BO and RL simulation variants differ internally because they were designed for different optimization tasks. Therefore, the numerical results of the BO and RL experiments should not be interpreted as directly comparable performance values across one identical simulation environment. The BO model evaluates static capacity configurations and includes buffer-based blocking effects, whereas the RL model uses explicit station queues, station-driven server components, and fixed station and worker capacities. The BO and RL findings should therefore be read as results for two related but task-specific simulation variants rather than as a direct quantitative comparison between both methods.

These modelling choices were necessary to create controlled optimization experiments, but they limit the direct transferability of the results to a real laboratory environment. A real testing laboratory may include additional constraints such as shift schedules, worker qualifications, machine-specific restrictions, maintenance, prioritization rules, and customer-specific service agreements. Future work could therefore calibrate the simulation model with historical operational data and validate the simulated KPIs against observed laboratory performance. This would make it possible to assess whether the optimized configurations and learned dispatching policies remain useful under more realistic operating conditions.

5.4.2 Objective and Reward Design

A further limitation is that both optimization approaches depend strongly on the definition of their objective or reward functions. For Bayesian Optimization, the selected capacity configurations are meaningful only relative to the chosen objective weights. Changing the weights changed the trade-off between service performance and resource efficiency, which shows that the optimization result is partly a consequence of the cost model. For Reinforcement Learning, the reward function similarly determines which dispatching behaviour is reinforced during training.

The weight settings and reward components used in this thesis were designed to create meaningful optimization tasks, but they are not the only possible formulations. In practical applications, the objective function should ideally be derived from real operational priorities and cost data. This could include personnel costs, station investment costs, service-level penalties, WIP costs, lateness penalties, and customer-specific priorities. Future work could perform a systematic sensitivity analysis of objective weights and reward components. Another possible extension would be to use multi-objective optimization instead of a weighted scalar objective, for example to approximate Pareto trade-offs between service performance and capacity costs.

5.4.3 Limitations of the Reinforcement Learning Formulation

The RL formulation used in this thesis was intentionally kept interpretable and tractable. The agent used tabular Q-learning with a discretized state representation and selected among four predefined dispatching rules. This design made the learned policy easy to analyse, but it also limited the range of behaviours the agent could discover. In particular, the agent did not select individual orders directly, but selected a heuristic rule that then chose the next order.

The action space could therefore be extended in future work. For example, Longest Waiting Time could be replaced or complemented by Longest Time in System, which may better capture accumulated flow time. The agent could also receive richer order attributes, such as processing-time estimates, due-date slack, failure history, expected downstream congestion, or priority classes. A more flexible action space could allow the agent to select specific orders rather than selecting among predefined heuristics. Such an extension would make the problem more complex, but it could also allow RL to learn behaviours that are not already encoded in the fixed dispatching rules.

Another limitation is that the RL agent controlled only the sorting station. This focused the experiment on one clear decision point, but it also restricted the potential benefit of dynamic control. Future work could investigate RL control at multiple stations or across the entire laboratory process. This could be implemented through a centralized agent, several station-specific agents, or a multi-agent RL formulation. Such extensions would allow the analysis of whether RL provides greater value when it can coordinate dispatching decisions across several bottlenecks.

The use of tabular Q-learning also limits scalability. A richer state space or direct order-selection action space would likely require alternative RL methods. Future work could therefore compare tabular Q-learning with methods such as SARSA (an on-policy temporal-difference method), deep Q-networks, actor-critic methods, or policy-gradient approaches. However, such methods should still be compared against strong domain-specific heuristics, because the results of this thesis show that simple but well-designed dispatching rules can be highly competitive.

5.4.4 Extensions of Bayesian Optimization

The BO experiments were limited to static capacity-configuration decisions within predefined parameter bounds. This was appropriate for evaluating BO as a black-box simulation optimizer, but it does not cover all possible simulation-optimization settings. Future work could extend the BO search space to include additional decision variables, such as shift patterns, worker allocation rules, buffer sizes, prioritization settings, or station-specific operating policies. This would make the optimization problem more realistic, but also more complex.

Another extension would be to evaluate robust or noise-aware Bayesian Optimization methods. The simulation outputs are stochastic, and each objective estimate depends on a finite number of replications. Methods that explicitly account for simulation noise could improve the reliability of selected configurations. It would also be useful to compare BO with additional optimization methods, such as evolutionary algorithms, simulated annealing, tree-structured Parzen estimators, or other sequential model-based optimization approaches. Such comparisons would help determine whether the observed BO performance is specific to the tested setup or generalizes to broader simulation-optimization problems.

5.4.5 Computational Constraints and Reproducibility

The experimental design was also constrained by computational resources. Several experiments required execution on the HPC infrastructure, and some Stage-1 tuning runs required many hours of runtime. The Stage-2 experiments with 10,000 training episodes were particularly demanding and in some cases required substantial memory resources. These constraints limited the number of repeated training runs, evaluation seeds, tuning variants, and sensitivity analyses that could be performed. They also limit the practical precision of the reported mean reward and objective values. Although values are reported with decimal places for consistency across tables, small differences at this level should not be interpreted as practically meaningful when the corresponding simulation outputs show substantial stochastic variation.

The use of fixed seeds, saved configurations, exported result files, and documented analysis scripts improves reproducibility. However, fully reproducing all optimization and RL tuning experiments remains computationally expensive. Future work with larger computational budgets could repeat complete tuning procedures across more independent training seeds and larger holdout sets. This would allow more precise estimates of variance and more robust conclusions about small performance differences between policies. Overall, the main limitations are not isolated implementation details, but the interaction between simulation assumptions, objective design, stochastic evaluation, algorithmic formulation, and computational budget.

6 Conclusion

This thesis applied and evaluated Bayesian Optimization and Q-learning-based Reinforcement Learning for different optimization tasks within a discrete-event simulation of a testing laboratory. The main objective was not to compare both methods as interchangeable optimization algorithms, but to investigate how they can support different decision levels within the same simulation-based case study. Bayesian Optimization was used for static simulation-configuration search and for Q-learning hyperparameter tuning, while Q-learning was used for dynamic dispatching decisions at the Sorting station. Overall, the results show that both methods can be applied meaningfully in this setting, but that their usefulness depends strongly on the type of decision problem, the objective formulation, and the evaluation protocol.

For the static capacity-configuration problem, Bayesian Optimization was able to identify high-performing configurations within the evaluated search budget. Under the default objective weights, the selected configurations were mainly service-oriented and used comparatively high capacities for several stations and for workers. This was consistent with the objective formulation, which strongly penalized late and incomplete orders. The comparison with Random Search showed that Bayesian Optimization achieved lower final best objective values descriptively under the default weights, but the difference was not statistically significant at the 5% level. Under the changed objective weights, Bayesian Optimization achieved significantly better final best objective values than Random Search. H1 was therefore only partially supported. In contrast, H2 was supported clearly, because changing the relative weights of service-related and capacity-related cost components led Bayesian Optimization to select substantially different capacity configurations. This shows that the optimization result is not independent of the objective function, but reflects the operational priorities encoded in the cost model.

For the dynamic dispatching problem, Q-learning was able to learn useful dispatching behaviour in the simulated laboratory environment. The manually configured fixed758 Q-learning agent outperformed the random agent and the FIFO, Earliest Due Date, and Longest Waiting Time baselines. However, it did not outperform the Highest Lateness Risk baseline. The final BO-tuned fixed758 Q-learning policy also did not show a statistically significant advantage over Highest Lateness Risk. H3 was therefore only partially supported. Overall, the RL results indicate potential for Q-learning-based adaptive dispatching, but they do not provide evidence that the learned policy clearly surpasses a strong domain-specific dispatching heuristic. This interpretation is important because the action space was deliberately limited to predefined dispatching rules, which made the policy interpretable but also constrained the possible improvement over the best fixed rule.

Bayesian Optimization-based hyperparameter tuning improved the Q-learning agent compared with the corresponding manually configured fixed758 agent. The improvement was statistically significant on the independent holdout seeds, which supports H4. At the same time, the practical magnitude of the improvement was small, and the reward distributions of several tuned variants overlapped substantially. The tuning results should therefore be interpreted as evidence that BO can improve a manually configured Q-learning setup, but not as evidence that it produced a uniquely dominant RL configuration. The staged tuning and holdout evaluation also showed that stochasticity and selection effects are important in simulation-based RL experiments. Independent holdout evaluation was therefore necessary to obtain a more conservative estimate of final policy performance.

The combined results support the view that Bayesian Optimization and Reinforcement Learning are complementary rather than directly competing approaches in simulation-based optimization. Bayesian Optimization was most suitable when the decision problem could be expressed as an expensive black-box parameter search, such as capacity optimization or RL hyperparameter tuning. Q-learning was relevant when decisions had to be made repeatedly during the simulation and depended on the current system state. This distinction between static configuration search and dynamic control is central to the interpretation of the thesis. The results therefore suggest that method selection in simulation optimization should be guided by the structure of the operational decision problem rather than by a general preference for one algorithmic approach.

The conclusions of this thesis are limited to the implemented simulation models, objective function, reward formulation, state and action representation, and computational budget. The findings should not be interpreted as directly transferable recommendations for a real testing laboratory

without further calibration and validation. Future work could use historical laboratory data, extend the BO search space, evaluate noise-aware or multi-objective optimization methods, and investigate richer RL formulations with expanded state and action spaces. In particular, RL control at multiple stations or direct order-selection policies could be examined to assess whether learning-based dispatching provides greater value in more flexible decision settings. Taken together, the thesis shows that discrete-event simulation can provide a useful experimental environment for combining Bayesian Optimization and Reinforcement Learning, provided that objectives, model assumptions, and evaluation procedures are defined carefully.

7 Appendix

7.1 Code Availability

The simulation and optimization scripts, visualization notebooks, configuration files, and experiment results are publicly available in the accompanying GitHub repository:
https://github.com/Dmfefnf/BA-Simulation-Optimization

7.2 Declaration of AI Assistance

In accordance with the ZHAW guidelines of the use of artificial intelligence (AI) in academic work, I declare that AI-based tools were used throughout the preparation of this report. The following LLM or systems were used between February and June 2026:

ChatGPT

- **Tool:** ChatGPT 5.5
- **Provider:** OpenAI (2026)
- **URL:** https://chatgpt.com/
- **Purpose in Project:**
  - Support with formulating text passages and linguistic revision to improve scientific phrasing
  - Provided help with formatting LaTeX Code for the scientific report
  - Proofreading and correction of the final report

Codex

- **Tool:** GPT-5.5
- **Provider:** OpenAI (2026)
- **URL:** https://openai.com/codex/
- **Purpose in Project:**
  - Assisted with the development and issue solving of code

References

Afshar, R. R., Rhuggenaath, J., Zhang, Y., & Kaymak, U. (2023). An Automated Deep Reinforcement Learning Pipeline for Dynamic Pricing. *IEEE Transactions on Artificial Intelligence*, *4*(3), 428–437. https://doi.org/10.1109/TAI.2022.3186292

Afshar, R. R., Vanschoren, J., Kaymak, U., Zhang, R., Wu, Y., Song, W., & Zhang, Y. (2026, March). Automated Reinforcement Learning: An Overview [arXiv:2201.05000 [cs.LG] version: 2]. https://doi.org/10.48550/arXiv.2201.05000

Amaran, S., Sahinidis, N. V., Sharda, B., & Bury, S. J. (2016). Simulation optimization: A review of algorithms and applications. *Annals of Operations Research*, *240*(1), 351–380. https://doi.org/10.1007/s10479-015-2019-x

Ax. (n.d.). Retrieved June 26, 2026, from https://ax.dev/

Barsce, J. C., Palombarini, J. A., & Martínez, E. C. (2018, May). Towards Autonomous Reinforcement Learning: Automatic Setting of Hyper-parameters using Bayesian Optimization [arXiv:1805.04748 [cs.AI]]. https://doi.org/10.48550/arXiv.1805.04748
Comment: Paper submitted to CLEI Electronic Journal. This is an extended version of the conference paper presented at Latin American Computer Conference (CLEI), 2017.

Beeks, M., Afshar, R. R., Zhang, Y., Dijkman, R., Dorst, C. v., & Looijer, S. d. (2022). Deep Reinforcement Learning for a Multi-Objective Online Order Batching Problem. *Proceedings of the International Conference on Automated Planning and Scheduling*, *32*, 435–443. https://doi.org/10.1609/icaps.v32i1.19829

Freitag, M., & Hildebrandt, T. (2016). Automatic design of scheduling rules for complex manufacturing systems by multi-objective simulation-based optimization. *CIRP Annals*, *65*(1), 433–436. https://doi.org/10.1016/j.cirp.2016.04.066

Gosavi, A. (2015). *Simulation-Based Optimization: Parametric Optimization Techniques and Reinforcement Learning* (Vol. 55). Springer US. https://doi.org/10.1007/978-1-4899-7491-4

Greasley, A., & Edwards, J. S. (2021). Enhancing discrete-event simulation with big data analytics: A review [_eprint: https://doi.org/10.1080/01605682.2019.1678406]. *Journal of the Operational Research Society*, *72*(2), 247–267. https://doi.org/10.1080/01605682.2019.1678406

Haeussler, S., & Netzer, P. (2020). Comparison between rule- and optimization-based workload control concepts: A simulation optimization approach. *International Journal of Production Research*, *58*(12), 3724–3743. https://doi.org/10.1080/00207543.2019.1634297

Letham, B., & Bakshy, E. (2019). Bayesian Optimization for Policy Search via Online-Offline Experimentation [Submitted 4/18; revised 3/19; published 9/19]. *Journal of Machine Learning Research*, *20*, 1–30. https://jmlr.org/papers/v20/18-225.html

Liu, P. (2023). *Bayesian Optimization: Theory and Practice Using Python*. Apress. https://doi.org/10.1007/978-1-4842-9063-7

Rabe, M., Dross, F., & Wuttke, A. (2017). Combining a Discrete-event Simulation Model of a Logistics Network with Deep Reinforcement Learning.

Shi, D., Fan, W., Xiao, Y., Lin, T., & Xing, C. (2020). Intelligent scheduling of discrete automated production line via deep reinforcement learning. *International Journal of Production Research*, *58*(11), 3362–3380. https://doi.org/10.1080/00207543.2020.1717008

van der Ham, R. (2023). Introduction — salabim 23.3.10 documentation. Retrieved December 10, 2025, from https://www.salabim.org/manual/Introduction.html

Wang, Z., & Liao, W. (2024). Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning [Num Pages: 2593-2610]. *Journal of Intelligent Manufacturing*, *35*(6), 2593–2610. https://doi.org/10.1007/s10845-023-02161-w

Wilson, A., Fern, A., & Tadepalli, P. (2014). Using Trajectory Data to Improve Bayesian Optimization for Reinforcement Learning [Submitted 1/12; revised 6/13; published 1/14]. *Journal of Machine Learning Research*, *15*, 253–282.

Yoo, W.-S., Kim, J., Concha, D. M., & Lee, C.-G. (2025). Optimizing Markov decision process state design for deep reinforcement learning manufacturing scheduling using Bayesian optimization. *Journal of Computational Design and Engineering*, *12*(10), 154–175. https://doi.org/10.1093/jcde/qwaf100

Zhang, H., Jiang, Z., & Guo, C. (2009). Simulation-based optimization of dispatching rules for semiconductor wafer fabrication system scheduling by the response surface methodology. *The International Journal of Advanced Manufacturing Technology*, *41*(1), 110–121. https://doi.org/10.1007/s00170-008-1462-0

Zmijewski, P., & Meseth, N. (2023). SimBO - A Framework For Simulation-Based Optimization Using Bayesian Optimization. *ECMS 2023 Proceedings edited by Enrico Vicario, Romeo*

Bandinelli, Virginia Fani, Michele Mastroianni, 222–228. https://doi.org/10.7148/2023-0222